



Introduction to optimization methods for training SciML models

Alena Kopaničáková, Elisa Riccietti

► To cite this version:

Alena Kopaničáková, Elisa Riccietti. Introduction to optimization methods for training SciML models. 2026. [⟨hal-05459235v1⟩](#)

HAL Id: hal-05459235

<https://hal.science/hal-05459235v1>

Preprint submitted on 15 Jan 2026 (v1), last revised 2 Jun 2026 (v2)

HAL is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers.

L'archive ouverte pluridisciplinaire **HAL**, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.



Distributed under a Creative Commons CC BY-NC-ND 4.0 - Attribution - Non-commercial use - No Derivative Works - International License

Introduction to optimization methods for training SciML models

Alena Kopaničáková* and Elisa Riccietti†

January 15, 2026

1 Introduction

Optimization is the foundation of modern machine learning (ML). Historically, optimization methods have been categorized by the degree to which they exploit derivative information. *First-order* methods rely exclusively on gradient evaluations, whereas (approximate) *second-order* methods incorporate curvature information via Hessians or suitable approximations. However, the massive scale of modern ML problems, combined with the principles of empirical risk minimization, has driven the field toward *stochastic optimization*. Stochastic optimization methods, such as Stochastic Gradient Descent (SGD) [73] and its variants (e.g., AdaGrad [18], and Adam [39]), employ inexpensive noisy gradient evaluations that scale efficiently with data size. Consequently, much of modern ML focuses on integrating first-order schemes, adaptive gradient methods, and curvature-aware techniques into stochastic optimization frameworks.

This familiar optimization landscape changes substantially in the context of *scientific machine learning* (SciML). Unlike classical ML, where abundant data allows stochastic approximation to perform well, SciML often operates in data-scarce regimes in which physical models must supplement, or even dominate, the available data. As a result, the optimization problems arising in SciML frequently take the form of *physics informed* or *operator constrained* formulations, in which the objective function incorporates a partial-differential equation (PDE) and boundary or initial conditions (BC or IC). These components change the structure of the objective function, since the loss depends on differential operators and induces global spatio-temporal coupling.

This difference has profound implications. In classical ML, the loss function typically decomposes into independent sample contributions, which enables efficient stochastic optimization. In contrast, SciML losses are often non-

*Toulouse-INP, IRIT-APO, ANITI, 2 Rue Charles Camichel, 31000 Toulouse, France; email: alena.kopanicakova@toulouse-inp.fr

†ENS de Lyon, CNRS, Inria, Université Claude Bernard Lyon 1, LIP, UMR 5668, 69342, Lyon cedex 07, France; email: elisa.riccietti@ens-lyon.fr

separable and globally coupled because differential operators introduce dependencies across the entire spatio-temporal domain. The resulting optimization landscape is typically highly *anisotropic* and *stiff*, with curvature and conditioning dominated by the spectra of the underlying differential operators rather than by the statistics of a dataset. Such losses may exhibit directions of substantial curvature, for example, those induced by high-order derivatives, alongside nearly flat directions corresponding to weakly identifiable components of the SciML model. This severely limits the effectiveness of first-order stochastic methods and often necessitates the use of deterministic or large-batch algorithms, as well as curvature-exploiting approaches, e.g., deterministic quasi-Newton methods.

At the same time, SciML also includes a rapidly growing class of *data-driven* formulations that more closely resemble classical supervised learning. Prominent examples include operator-learning models [52], neural surrogate models trained on high-fidelity PDE simulation data [62], autoencoder-based reduced-order models that learn low-dimensional latent dynamics [46], or neural differential equation frameworks that fit observational or simulated trajectories [9]. Because their training objectives decompose over samples and admit mini-batching, these models inherit much of the optimization behavior of standard ML and benefit directly from well-established stochastic optimization techniques. As a result, effective optimization methods in SciML span two complementary regimes: physics-constrained settings and data-driven settings that align more closely with large-scale stochastic ML training.

In this document, we first introduce the optimization problems that arise in both traditional ML and SciML, highlighting their structural differences and analytical properties. We then present a foundational overview of deterministic and stochastic optimization methods, explaining how these algorithms can be adapted to address the specific challenges posed by SciML models. Given the breadth of the optimization field, the material covered here does not aim to be exhaustive. Instead, we present a concise introduction to common optimization techniques, demonstrate how they can be adapted to SciML training through practical examples¹, and point toward promising research directions.

2 Empirical risk minimization and the finite sum minimization problem

In modern ML, a model is characterized by a parameter vector $\theta \in \mathbb{R}^n$ and a parameterized mapping $h_\theta : \mathcal{X} \rightarrow \mathcal{Y}$, where $\mathcal{X}$ denotes the input (feature) space and $\mathcal{Y}$ the output space. The mapping h_θ transforms an input sample $x \in \mathcal{X}$ into an output prediction $h_\theta(x) \in \mathcal{Y}$, and its form is typically defined by a deep neural network (DNN) whose parameters are collected in the vector θ .

We aim to find a model h_θ that provides the best possible approximation of the desired mapping. The process of searching for such a model is called

¹The accompanying notebooks are available at https://github.com/kopanicakova/intro_opt_SciML.git.

training and involves solving an optimization problem defined by an expected loss over the data distribution. Let (x, y) denote a generic data pair drawn from an unknown probability distribution $\mathcal{P}$ on $\mathcal{X} \times \mathcal{Y}$. Let $\ell : \mathcal{Y} \times \mathcal{Y} \rightarrow \mathbb{R}_+$ denote a loss function measuring the model’s performance. For example, ℓ may represent a prediction error (supervised learning), a reconstruction error (autoencoders), a divergence between distributions (generative models), or the violation of physical laws (SciML).

The general training objective is to find the parameters θ that minimize the *expected risk* of the model, given by

$$R(\theta) = \int_{\mathcal{X} \times \mathcal{Y}} \ell(h_\theta(x), y) d\mathcal{P}(x, y) = \mathbb{E}_{(x, y) \sim \mathcal{P}}[\ell(h_\theta(x), y)]. \quad (1)$$

However, since the distribution $\mathcal{P}$ is usually unknown in practice, learning relies on a finite dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^m$ consisting of m samples drawn independently and identically from $\mathcal{P}$, i.e., $(x_i, y_i) \stackrel{\text{i.i.d.}}{\sim} \mathcal{P}$.

Using the dataset $\mathcal{D}$, the expected risk (1) can be approximated, for example, using a Monte-Carlo estimate, yielding the *empirical risk*

$$\hat{R}_m(\theta) = \frac{1}{m} \sum_{i=1}^m \ell(h_\theta(x_i), y_i).$$

Under standard assumptions (i.i.d. sampling, bounded variance), $\hat{R}_m(\theta)$ converges to $R(\theta)$ as $m \rightarrow \infty$; see, e.g., [87, 77].

The *Empirical Risk Minimization (ERM)* principle consists of minimizing $\hat{R}_m$, which gives rise to the canonical *finite-sum optimization problem*

$$\min_{\theta \in \mathbb{R}^n} f(\theta) := \frac{1}{m} \sum_{i=1}^m f_i(\theta), \quad \text{where} \quad f_i(\theta) = \ell(h_\theta(x_i), y_i). \quad (2)$$

Here, each function $f_i(\theta)$ represents the contribution of one sample (x_i, y_i) to the total objective $f(\theta)$.

2.1 Examples of ERM across learning paradigms

In this section, we illustrate how the abstract ERM formulation translates into practical learning settings by presenting three representative examples.

2.1.1 Example 1: Supervised learning - classification

In supervised learning, we are given a labeled dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^m$, where each input vector $x_i \in \mathbb{R}^d$ is associated with an output label y_i indicates affiliation to a given class. The goal is to learn the parameters $\theta \in \mathbb{R}^n$ of the model $h_\theta : \mathbb{R}^d \rightarrow \mathbb{R}$ that predicts the label $y_i \in \{0, 1\}$ for a given input x_i . For simplicity, we restrict our attention to the binary classification setting.

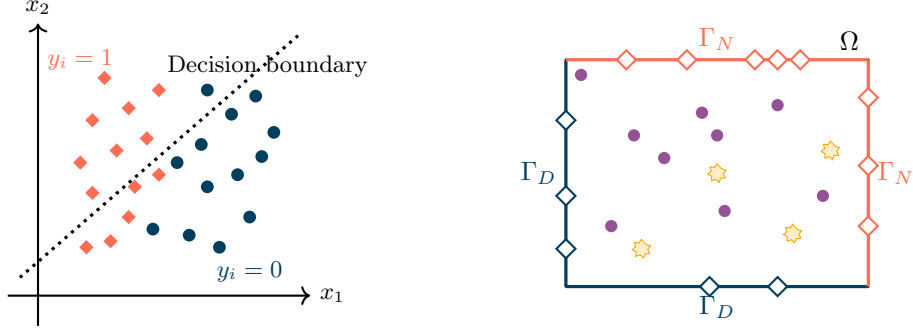


Figure 1: Left: Example of binary classification in the (x_1, x_2) -plane: Samples from two classes $y_i \in \{0, 1\}$ are shown as orange diamonds and blue circles, respectively. The dotted line indicates the decision boundary separating the two regions. Right: Example of a computational domain Ω with sampled collocation points: Dirichlet boundaries Γ_D (blue), Neumann boundaries Γ_N (orange), interior points $\mathcal{D}_\Omega$ (purple), and optional empirical data samples $\mathcal{D}_{\text{data}}$ (yellow).

Logistic regression (linear model) In logistic regression, the model is

$$h_\theta(x_i) = \rho(x_i^\top \theta),$$

where $\rho(t) = \frac{1}{1+e^{-t}}$ is the *Sigmoid* activation. The discrepancy between prediction and label is measured by the logistic loss, given as

$$\ell(h_\theta(x_i), y_i) = -y_i \log h_\theta(x_i) - (1 - y_i) \log(1 - h_\theta(x_i)). \quad (3)$$

The ERM objective then becomes

$$\hat{R}_m(\theta) = \frac{1}{m} \sum_{i=1}^m \left[-y_i \log \rho(x_i^\top \theta) - (1 - y_i) \log(1 - \rho(x_i^\top \theta)) \right].$$

This function is smooth and (strictly) *convex*, and therefore its minimization admits a unique minimizer. The practical difficulty of minimizing $\hat{R}_m$ depends predominantly on the geometry of the data. If the data are well-conditioned and the classes are separable, convergence is fast. If, on the other hand, the samples are nearly collinear or the classes overlap, the optimization problem becomes more challenging.

Deep neural network (nonlinear model) If the data are not linearly separable, linear models may be insufficient to capture the underlying structure. In such cases, DNNs provide flexible nonlinear alternatives capable of learning complex nonlinear relationships and feature interactions present in the data.

Let h_θ denote a feed-forward DNN composed of weights $U_1, U_2, \dots, U_Q$ and bias vectors $b_1, b_2, \dots, b_Q$, which can be collected into a parameter vector θ as

$\theta = \text{vec}(U_1, \dots, U_Q, b_1, \dots, b_Q) \in \mathbb{R}^n$. A standard Q -layer network is defined recursively as

$$z_0 = x, \quad z_j = \rho(U_j z_{j-1} + b_j), \quad j = 1, \dots, Q-1,$$

with the output

$$h_\theta(x) = \rho(U_Q z_{Q-1} + b_Q). \quad (4)$$

Here, ρ denotes an activation function of the user's choice, e.g., *ReLU*, or *Tanh*.

Given a dataset $\mathcal{D}$ and a loss function ℓ , the empirical risk is now given as

$$\hat{R}_m(\theta) = \frac{1}{m} \sum_{i=1}^m \ell(h_\theta(x_i), y_i).$$

The choice of the loss function ℓ depends on the task at hand. For example, in a binary classification problem, one can use the logistic loss (3). Here, we emphasize that the use of DNNs makes $\hat{R}_m$ highly nonconvex. The complexity of the resulting optimization problem depends strongly on the data geometry and the chosen DNN architecture.

2.1.2 Example 2: Supervised learning - regression

Regression is a supervised learning paradigm widely used in data-driven SciML. Given a dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^m$, the goal is to predict a real-valued output $y_i \in \mathbb{R}$ from a given input x_i . For example, the input may consist of observational measurements, while the output might represent the solution of a PDE or a quantity derived from it.

In regression, the discrepancy between the model prediction and the true label is commonly measured using the squared ℓ_2 loss. Defining the stacked data labels $y := [y_1, \dots, y_m]^\top$ and model outputs $u(\theta) := [h_\theta(x_1), \dots, h_\theta(x_m)]^\top$, the ERM problem takes on the following form:

$$\hat{R}_m(\theta) = \frac{1}{2m} \|u(\theta) - y\|^2. \quad (5)$$

This ERM formulation is commonly referred to as a *least-squares* problem.

If the labels depend linearly on the inputs, this relationship can be modelled using the linear model $h_\theta(x_i) = x_i^\top \theta$. In this particular case, the ERM problem admits the compact representation $\hat{R}_m(\theta) = \frac{1}{2m} \|X\theta - y\|^2$, where $X \in \mathbb{R}^{m \times d}$ denotes the data matrix. Minimizing this objective corresponds to solving a *linear least-squares* problem. Thus, if X has full column rank, the solution is given in closed form as $\theta = (X^\top X)^{-1} X^\top y$.

If, on the other hand, the model h_θ is nonlinear, e.g., a DNN introduced given by (4), the ERM problem (5) becomes a *nonlinear least-squares* problem. Here, the term “nonlinear” refers to the nonlinear dependence of $h_\theta(x)$ on θ , even though the loss remains quadratic in the residual (misfit). The resulting optimization problem is generally *nonconvex* and may admit multiple local minima and saddle points, making its minimization considerably challenging.

2.1.3 Example 3: Physics-informed neural networks (PINNs)

In physics-informed learning, our goal is to approximate a solution of a differential equation using an ML model. Let $\Omega \subset \mathbb{R}^d$ be a bounded domain with boundary $\partial\Omega = \Gamma_D \cup \Gamma_N$, where Γ_D and Γ_N represent Dirichlet and Neumann parts, respectively. We consider the following PDE:

$$\begin{aligned}\mathcal{N}[u](x) &= q(x), & x \in \Omega, \\ u(x) &= g_D(x), & x \in \Gamma_D, \\ \partial_n u(x) &= g_N(x), & x \in \Gamma_N,\end{aligned}\tag{6}$$

where $\mathcal{N}$ is a differential operator, q is a source term, and $\partial_n u$ denotes the normal derivative on Γ_N . The goal is to approximate the unknown solution $u : \Omega \rightarrow \mathbb{R}$ with a DNN $h_\theta : \Omega \rightarrow \mathbb{R}$.

In the continuous setting, model quality is quantified by an expected risk functional that penalizes the PDE residual, boundary condition violations, and mismatch with available data, i.e.,

$$\begin{aligned}R(\theta) &= \gamma_\Omega \int_\Omega |\mathcal{N}[h_\theta](x) - q(x)|^2 d\mathcal{P}_\Omega(x) + \gamma_D \int_{\Gamma_D} |h_\theta(x) - g_D(x)|^2 d\mathcal{P}_{\Gamma_D}(x) \\ &\quad + \gamma_{\Gamma_N} \int_{\Gamma_N} |\partial_n h_\theta(x) - g_N(x)|^2 d\mathcal{P}_{\Gamma_N}(x) \\ &\quad + \gamma_{\Omega_{\text{data}}} \int_{\Omega_{\text{data}}} |h_\theta(x) - y(x)|^2 d\mathcal{P}_{\text{data}}(x),\end{aligned}\tag{7}$$

where $\gamma_\Omega, \gamma_D, \gamma_{\Gamma_N}, \gamma_{\Omega_{\text{data}}} \in \mathbb{R}^+$.

In practice, the integrals in (7) must be approximated. A common approach is to use Monte-Carlo (MC) quadrature. Thus, we use the following datasets of randomly sampled collocation points

$$\begin{aligned}\mathcal{D}_\Omega &= \{x_j^{(\Omega)}\}_{j=1}^{m_\Omega}, & \mathcal{D}_{\Gamma_D} &= \{x_r^{(D)}\}_{r=1}^{m_D}, \\ \mathcal{D}_{\Gamma_N} &= \{x_k^{(N)}\}_{k=1}^{m_N}, & \mathcal{D}_{\text{data}} &= \{(x_i^{(\text{data})}, y_i)\}_{i=1}^{m_{\text{data}}},\end{aligned}$$

to obtain the following empirical risk formulation:

$$\begin{aligned}\hat{R}_m(\theta) &= \frac{\gamma_\Omega}{m_\Omega} \sum_{j=1}^{m_\Omega} |\mathcal{N}[h_\theta](x_j^{(\Omega)}) - q(x_j^{(\Omega)})|^2 + \frac{\gamma_D}{m_D} \sum_{r=1}^{m_D} |h_\theta(x_r^{(D)}) - g_D(x_r^{(D)})|^2 \\ &\quad + \frac{\gamma_N}{m_N} \sum_{k=1}^{m_N} |\partial_n h_\theta(x_k^{(N)}) - g_N(x_k^{(N)})|^2 + \frac{\gamma_{\text{data}}}{m_{\text{data}}} \sum_{i=1}^{m_{\text{data}}} |h_\theta(x_i^{(\text{data})}) - y_i|^2.\end{aligned}$$

Here, we point out that more efficient quadrature techniques than MC can be used in order to reduce the approximation error. For instance, *quasi-Monte Carlo* (QMC) methods based on low-discrepancy sequences (e.g., Sobol') can achieve approximation errors of order $\mathcal{O}(m^{-1}(\log m)^d)$, compared to the $\mathcal{O}(m^{-1/2})$ rate of standard MC under suitable regularity assumptions [16]. Here,

we emphasise that increasing the number of collocation points in PINNs is *not* analogous to adding independent samples in classical ML, as the same deterministic PDE residual is evaluated at additional locations. Due to the global coupling induced by the differential operator, nearby points are often strongly correlated, so oversampling does not yield proportional accuracy gains [63, 53].

As before, in order to find optimal parameters of model h_θ , we minimize $\widehat{R}_m$. Although h_θ is usually a standard DNN, the choice of architecture requires particular care because $\widehat{R}_m$ involves *spatial derivatives* of h_θ . In particular, the activation function ρ must be sufficiently smooth to allow stable differentiation through the PDE operator. To this aim, smooth activations such as *Tanh* or *Sigmoid*, or their adaptive variants [33] are often used.

2.2 Enhancing the problem definition

The minimization problems discussed above (classical data-driven and physics-informed) differ significantly in their mathematical structure. Nevertheless, in both settings, our goal is to minimize a highly *nonconvex* objective while ensuring that the learned model generalizes well beyond the training samples/collocation points. To this aim, it is often beneficial to refine or augment the objective function f . Such modifications can improve generalization, enhance the well-posedness of the optimization problem, and lead to more stable and efficient training dynamics. The following subsections highlight two key directions for improving the problem definition: regularization techniques, and strategies for balancing heterogeneous loss terms.

2.2.1 Regularization

Regularization techniques augment the objective with an additional term $\widehat{R}_{\text{reg}}$, leading to the following modified objective function:

$$f(\theta) := \widehat{R}_m(\theta) + \gamma \widehat{R}_{\text{reg}}(\theta),$$

where $\gamma > 0$ is a regularization parameter. A common strategy is to incorporate ℓ_2 or ℓ_1 penalties on the parameters or on the network’s output gradients:

$$\widehat{R}_{\text{reg}}(\theta) = \|\theta\|^2, \quad \widehat{R}_{\text{reg}}(\theta) = \|\nabla_\theta h_\theta\|^2.$$

The first term encourages smoother parameter trajectories, while the second directly penalizes sharp variations in the network output.

In PINNs, one can additionally employ physics-guided regularization to control high-frequency components. A frequently used formulation is

$$\widehat{R}_{\text{reg}}(\theta) = \int_{\Omega} |\nabla^p h_\theta(x)|^2 dx,$$

where p is the differential order of the operator $\mathcal{N}$. Such derivative- or spectrum-based penalties enforce physically consistent smoothness and often improve both training stability and out-of-distribution generalization.

2.2.2 Balancing the loss terms

As we have seen, the objective function f is frequently composed of N terms as $f(\theta) = \sum_{i=1}^N \gamma_i \hat{R}_i(\theta)$, where $\gamma_i \in \mathbb{R}^+$. Selecting appropriate weights $\gamma = (\gamma_1, \dots, \gamma_N)$ is often challenging, as overly large values may dominate the optimization and distort the geometry of the composite loss, whereas overly small values may fail to effectively enforce the corresponding terms, such as regularization components in standard ML or PDE-related constraints in PINNs. To alleviate this difficulty, loss-balancing strategies aim to rescale the contributions of each component so that gradients and curvature are more homogeneous.

Bilevel approaches A principled strategy for choosing the weights γ is to treat them as *hyperparameters* that are selected by optimizing some outer performance measure $\mathcal{C}$, such as the validation loss or the mean PDE residual. This leads to the following bilevel optimization problem:

$$\min_{\gamma > 0} \mathcal{C}(\theta(\gamma)), \quad \text{s.t.} \quad \theta(\gamma) = \arg \min_{\theta} \sum_{i=1}^N \gamma_i \hat{R}_i(\theta),$$

where the inner problem determines the model parameters θ for a fixed weights γ , while the outer problem adjusts γ based on the performance measure $\mathcal{C}$.

At the inner optimum $\theta^*(\gamma)$, the stationarity condition

$$\nabla_{\theta} \hat{R}_m(\theta^*, \gamma) = 0$$

implicitly defines the dependence of $\theta^*(\gamma)$ on γ . Differentiating this condition with respect to γ yields the *hypergradient* $\nabla_{\gamma} \mathcal{C}$ whose components are given as

$$\frac{\partial \mathcal{C}}{\partial \gamma_j} = -(\nabla_{\theta} \mathcal{C}(\theta^*))^{\top} H^{-1}(\theta^*, \gamma) \nabla_{\theta}^2 \hat{R}_j(\theta^*), \quad H = \nabla_{\theta} \hat{R}_m(\theta^*, \gamma),$$

which shows that the response of θ^* to changes in γ is governed by the inverse Hessian H^{-1} of the inner objective.

In principle, this formula provides an exact update for the loss weights γ . In practice, however, computing or inverting the Hessian H is prohibitive, especially for large ML models. Therefore, practical loss-balancing algorithms often rely on inexpensive approximations or heuristics.

Adaptive weighting strategies Adaptive weighting strategies are heuristic techniques that can be used to adjust the weights γ so that the different loss components contribute more evenly during the optimization process. The common approach is to employ *global heuristics*, that assign a single coefficient to each loss component. One of the most widely used approaches in this category is *Gradient-norm balancing (GradNorm)* [11], which updates the weights γ in order to equalize the magnitudes of the per-loss gradients with respect to the shared parameters. At each iteration, *GradNorm* computes for each component i the partial gradients $\|\gamma_i \nabla_{\theta} \hat{R}_i(\theta)\|$, measuring the contribution of the i -th

loss component to the overall gradient. A target value for these quantities is then derived from the relative training speed of the losses:

$$r_i = \frac{\widehat{R}_i(\theta)/\widehat{R}_i(0)}{\frac{1}{N} \sum_{j=1}^N \widehat{R}_j(\theta)/\widehat{R}_j(0)}.$$

GradNorm then minimizes the surrogate objective

$$L_{\text{grad}}(\gamma) = \sum_{i=1}^N \left| \|\gamma_i \nabla_{\theta} \widehat{R}_i(\theta)\| - \frac{1}{N} \sum_{j=1}^N \|\gamma_j \nabla_{\theta} \widehat{R}_j(\theta)\| r_i^{\zeta} \right|,$$

with respect to the weights γ , where $\zeta > 0$ controls how strongly the method enforces equalized relative descent rates among the losses. In practice, this step is inexpensive as only a single gradient-descent update is applied to each γ_i using $\partial L_{\text{grad}}/\partial \gamma_i$, followed by a normalization step to maintain weights on a similar scale. This procedure equalizes the influence of the different loss terms without computing second-order information, thus providing a practical and efficient approximation to bilevel loss balancing. Other heuristics include for example uncertainty-based weighting [37, 96] and stage-dependent curricula learning [90].

Using a single global weight for each loss component presumes that all samples contribute comparably. However, in PINNs, the PDE residual is often highly non-uniform, frequently differing by several orders of magnitude across the domain [59]. This mismatch can cause the optimizer to overfit well-resolved regions while failing to sufficiently reduce the error in stiff or under-resolved parts of the domain. *Spatially adaptive weighting* strategies were designed to mitigate this issue by assigning location-dependent weight ξ_j to all collocation points, thus replacing the standard residual term by

$$\widehat{R}_{\Omega}(\theta, \xi) = \frac{1}{m_{\Omega}} \sum_{j=1}^{m_{\Omega}} \xi_j |\mathcal{N}[h_{\theta}](x_j) - q(x_j)|^2, \quad \sum_{j=1}^{m_{\Omega}} \xi_j = m_{\Omega}, \quad \xi_j \geq 0.$$

A widely used approach for constructing spatially adaptive weights ξ is residual-based weighting [4, 10], which constructs the weights directly from the pointwise residual $r_j(\theta) = \mathcal{N}[h_{\theta}](x_j) - q(x_j)$. A typical example includes power-law normalization, where the j -th weight takes on the following form:

$$\xi_j = \frac{|r_j|^{\beta}}{\frac{1}{m_{\Omega}} \sum_{k=1}^{m_{\Omega}} |r_k|^{\beta}}, \quad \beta > 0.$$

This allows us to emphasize under-resolved regions and encourage a more uniform residual reduction across the domain Ω .

3 Solving the optimization problem

Given a nonlinear, differentiable objective (loss) function $f : \mathbb{R}^n \rightarrow \mathbb{R}$, the goal of numerical optimization is to solve

$$\min_{\theta \in \mathbb{R}^n} f(\theta),$$

which is often referred to as *training*. Ideally, we would like to obtain the *global minimizer* of the function, that is, a point in which the function reaches the lowest possible value (assuming f is bounded from below). In practice, it is very difficult to reach such a point, and the usual optimization methods rather reach a *stationary point*, i.e., a point in which the gradient is zero. In most cases, this stationary point will be a *local minimizer*, a point such that the function does not decrease in its neighbourhood. However, it is usually not possible to exclude it from being a *saddle point*.

To reach a stationary point, one typically employs *iterative algorithms*, also referred to as *optimizers*. Starting from an initial guess θ_0 , such methods generate a sequence $\{\theta_k\}_{k \in \mathbb{N}}$, where k denotes the iteration index, that converges to a stationary point θ^* through updates of the form

$$\theta_{k+1} = \theta_k + \alpha_k p_k,$$

where $p_k \in \mathbb{R}^n$ denotes a search direction and $\alpha_k > 0$ is the step size, often referred to as the learning rate.

Assuming that f is regular enough, in order to build search directions, most optimization methods employ the derivatives of f , i.e., the *gradient* $\nabla f(x)$ and the *Hessian* $H(x) \in \mathbb{R}^{n \times n}$. Based on the information they use, optimization methods are classically divided into three large classes:

1. *First-order methods*: These methods use only gradient information. As a consequence, their iteration cost is low, but they may require many iterations to converge. The most notable example is the gradient descent (GD) method, whose iterates are defined as follows:

$$\theta_{k+1} = \theta_k - \alpha_k \nabla f(\theta_k), \tag{8}$$

for $\alpha_k > 0$.

2. *Second-order methods*: These methods incorporate curvature information from the Hessian or its approximations, providing much faster local convergence at a higher per-iteration cost. The most notable example is Newton's method, whose iterates are formally defined as follows:

$$\theta_{k+1} = \theta_k - \alpha_k H(\theta_k)^{-1} \nabla f(\theta_k).$$

3. *Adaptive (preconditioned) gradient methods*: A central challenge in large-scale ML optimization is balancing the efficiency of first-order methods with the curvature-awareness of second-order methods. Adaptive gradient methods offer a trade-off by constructing inexpensive, per-parameter curvature approximations using only first-order information. Let M_k^{-1} be a (possibly data-dependent) linear transformation or preconditioner, the adaptive gradient methods have the following form:

$$\theta_{k+1} = \theta_k - \alpha_k M_k^{-1} \nabla f(\theta_k), \tag{9}$$

where $M_k = I$ recovers standard GD, and the formal choice $M_k = H(\theta_k)$ corresponds to a Newton’s step. Practical optimizers such as AdaGrad [18], or Adam [39] utilize (9) with a diagonal preconditioner M_k , which is built from running estimates of first and/or second moments of past gradients.

3.1 Neural tangent kernel

The Neural Tangent Kernel (NTK) is a theoretical tool introduced in [32] to study the training dynamics of neural networks, especially in the “infinite-width” limit. It connects neural network training with kernel methods, providing a framework for understanding how gradient descent shapes the function learned by a DNN. In particular, the eigenvalue spectrum of the NTK governs the dynamics and conditioning of first-order optimization methods and provides insights into the training behavior of DNNs. For PINNs, NTK can be used to show that the optimization problem inherits stiffness from the underlying differential operator $\mathcal{N}$, as well as structural biases induced by the network architecture. These effects make training notoriously challenging and often render first-order methods such as GD inefficient.

3.1.1 Function-space view: GD as kernel flow

In this section, we introduce the NTK from a function-space viewpoint. Let $h_\theta : \Omega \rightarrow \mathbb{R}$ be a DNN with parameters $\theta \in \mathbb{R}^n$. During training via GD, the network updates the parameters, and the *NTK* describes how these parameter updates translate into changes in the network output.

Continuous-time kernel dynamics Continuous GD updates the parameters according to

$$\dot{\theta}(t) = -\nabla_\theta f(\theta(t)),$$

where $\dot{\theta}(t)$ denotes the derivatives of $\dot{\theta}$ with respect to t . We are interested in the induced evolution of the function h_θ . Using the chain rule, we get

$$\dot{h}_\theta(x) = \nabla_\theta h_\theta(x)^\top \dot{\theta}(t) = -\nabla_\theta h_\theta(x)^\top \nabla_\theta f(\theta). \quad (10)$$

If we consider the gradient of the loss, we have

$$\nabla_\theta f = \int_\Omega \nabla_\theta h_\theta(x') \frac{\delta f}{\delta h(x')} dx', \quad (11)$$

where $\delta f / \delta h(x')$ is the functional derivative of f with respect to $h(x')$. Combining (10) with (11) gives

$$\dot{h}_\theta(x) = - \int_\Omega K_\theta(x, x') \frac{\delta f}{\delta h(x')} dx',$$

where

$$K_\theta(x, x') = \nabla_\theta h_\theta(x)^\top \nabla_\theta h_\theta(x')$$

is the *NTK*. Thus, when GD updates the parameters θ , the corresponding evolution of the function h_θ is governed by the kernel K_θ . In other words, *GD in parameter space induces a kernel gradient flow in function space*.

Discrete-time NTK dynamics The same argument can be developed in a discretized setting, leading to the empirical NTK. This can be simply shown by considering the empirical squared loss, given in (5). In this case, the GD step with fixed step size $\alpha > 0$ reads as

$$\theta_{k+1} = \theta_k - \alpha \nabla f_m(\theta_k) = \theta_k - \frac{\alpha}{m} J_k^\top (u(\theta_k) - y), \quad (12)$$

where

$$J_k = \begin{bmatrix} \nabla_\theta h_{\theta_k}(x_1)^\top \\ \vdots \\ \nabla_\theta h_{\theta_k}(x_m)^\top \end{bmatrix} \in \mathbb{R}^{m \times n}$$

is the Jacobian of h_{θ_k} evaluated at all data points.

Following the linearization argument of [32, 47], we expand h_θ around θ_k :

$$h_{\theta_{k+1}}(x) \approx h_{\theta_k}(x) + \nabla_\theta h_{\theta_k}(x)^\top (\theta_{k+1} - \theta_k).$$

Since in our notation the predicted outputs evaluated on the training set satisfy $u(\theta_k) = h_{\theta_k}(x_{1:m})$, the linearization directly applies to $u(\theta_k)$ as well. Substituting the GD update (12), into this linearization then yields an approximate evolution equation for the predicted outputs:

$$u(\theta_{k+1}) \approx u(\theta_k) - \alpha \Theta_k (u(\theta_k) - y), \quad \Theta_k := \frac{1}{m} J_k J_k^\top, \quad (13)$$

where $\Theta_k \in \mathbb{R}^{m \times m}$ is the empirical NTK at iteration k .

This indicates that training evolves like kernel gradient descent in the reproducing kernel Hilbert space (RKHS) induced by the NTK. In sufficiently wide neural networks or in the almost stationary regime (α small), the NTK remains nearly constant during training, providing a tractable mathematical description of deep learning dynamics.

3.1.2 Parameter-space viewpoint

The function-space analysis above can be complemented by a parameter-space perspective. For instance, for the empirical least-square loss (5), the gradient and Hessian are given as

$$\nabla f_m(\theta_k) = \frac{1}{m} J_k^\top (u(\theta_k) - y), \quad H(\theta_k) = \frac{1}{m} J_k^\top J_k + (\text{higher-order terms}).$$

If we are in the NTK regime, i.e., the network is wide and approximately linear in θ , the higher-order terms are small, and H can be approximated by the Gauss-Newton (GN) approximation $\frac{1}{m} J_k^\top J_k$. Since the empirical NTK is

$\Theta_k = \frac{1}{m} J_k J_k^\top$, it follows that $H(\theta_k)$ and Θ_k share the same nonzero eigenvalues. This creates a direct correspondence between NTK conditioning in function space and Hessian conditioning in parameter space. In particular, it implies that small eigenvalues of Θ_k imply small eigenvalues of $H(\theta_k)$, so that flat directions in the objective function landscape correspond exactly to slow-learning directions in function space.

As a consequence, if the NTK and the Hessian are ill-conditioned, this negatively impacts the convergence of the first-order methods. For example, let us consider the GD iteration (8) applied to a strongly convex quadratic model with constant Hessian $H = \nabla^2 f(\theta)$ and step size $0 < \alpha < 2/\lambda_{\max}(H)$. It is well known that the GD iterates satisfy the following classical contraction bound (see, e.g., [67, Section 3.2]):

$$\|\theta_{k+1} - \theta^*\| \leq \left(\frac{\kappa(H) - 1}{\kappa(H) + 1} \right) \|\theta_k - \theta^*\|, \quad \kappa(H) = \frac{\lambda_{\max}(H)}{\lambda_{\min}(H)}.$$

Thus, if $\lambda_{\min}(H)$ is very small, the contraction factor approaches one and progress becomes arbitrarily slow. Similar observations extend to stochastic gradient methods, which we discuss in the upcoming sections.

3.2 Explicit mode-wise error decay and spectral bias

The eigenvalues of Θ_k determine the contraction rates of the corresponding error modes during GD training. To understand this behaviour, let us again consider empirical least-squares loss (5). The function-space error is defined as $e_k := u(\theta_k) - y$. Using NTK-induced update (13), we can obtain a linear recursion of the following form:

$$e_{k+1} \approx (I - \alpha \Theta_k) e_k. \tag{14}$$

In regimes where Θ_k varies slowly, we may approximate Θ_k locally with constant Θ , i.e., $\Theta_k \approx \Theta$, yielding

$$e_k \approx (I - \alpha \Theta)^k e_0,$$

which is the standard linear kernel gradient-flow approximation used in [32, 47].

Let the eigendecomposition of Θ be

$$\Theta = Q \Lambda Q^\top, \quad \Lambda = \text{diag}(\lambda_1, \dots, \lambda_m),$$

where q_i denotes the i -th eigenvector (column of Q) and λ_i is the associated eigenvalue. Because $\{q_i\}_{i=1}^m$ form an orthonormal basis of $\mathbb{R}^m$, we may expand the initial error as $e_0 = \sum_{i=1}^m c_i q_i$. Applying the recursion $e_{k+1} = (I - \alpha \Theta_k) e_k$ repeatedly, then yields

$$e_k = \sum_{i=1}^m c_i (1 - \alpha \lambda_i)^k q_i \quad \text{and} \quad \|e_k\|^2 = \sum_{i=1}^m c_i^2 (1 - \alpha \lambda_i)^{2k}.$$

Thus, each coefficient c_i associated with eigenvector q_i is multiplied by the scalar factor $(1 - \alpha \lambda_i)$ at every iteration. This implies that the i -th component decays

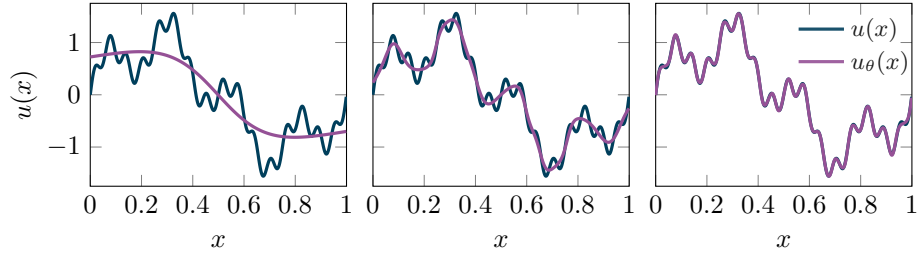


Figure 2: Example of spectral bias to learn target function u_θ (purple) composed of low-, medium-, and high-frequencies. The network prediction u_θ (purple) is reported at training iterations 100 (left), 1500 (middle), and 7500 (right).

geometrically at rate $(1 - \alpha\lambda_i)^k$. Hence, if λ_i is large, the factor $1 - \alpha\lambda_i$ is significantly below one, which leads to a rapid decrease of the corresponding error mode. In contrast, if λ_i is very small, then $1 - \alpha\lambda_i \approx 1$, so the contraction per iteration is extremely small.

This imbalance in decay rates across different eigenmodes is referred to as *spectral bias* [70, 93]. DNNs learn components of the solution that align with large NTK eigenvalues (typically low-frequency or smooth structures) much faster than the components associated with small eigenvalues (often high-frequency components). As a consequence, the smallest NTK eigenvalue $\lambda_{\min}(\Theta)$ governs the slowest decaying error and directly controls the overall speed of learning.

Here, we emphasize that different DNN architectures induce different NTKs and, therefore, exhibit different implicit biases. Specialized architectures can be designed to mitigate spectral bias; see, for example, Fourier features [84].

Numerical example #1: To illustrate the spectral bias phenomenon in practice, we consider nonlinear regression as specified in Section 2.1.2 to learn function that is a sum of sinusoids of increasing frequency, i.e., $u(x) = \sin(2\pi x) + 0.5 \sin(8\pi x) + 0.2 \sin(32\pi x)$. The training is performed using the GD method. As we can see from Figure 2, in early training (iteration 100), the network fits only the low frequencies. Medium frequencies appear later (iteration 1500), and only after many more iterations, the network begins to capture the high-frequency oscillations (iteration 7500).

3.3 Kernels of adaptive and second-order methods

We have seen that the conditioning of the NTK and the Hessian plays a central role in determining the training dynamics of the GD method. A similar analysis extends to adaptive and second-order methods. Using the same linearization as in (13), and writing $g_k = \nabla f_m(\theta_k) = \frac{1}{m} J_k^\top (u(\theta_k) - y)$, the update rule (9) gives

$$\theta_{k+1} - \theta_k = -\frac{\alpha_k}{m} M_k^{-1} J_k^\top (u(\theta_k) - y),$$

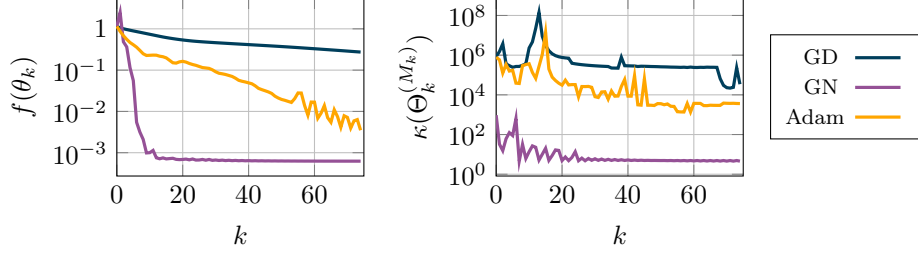


Figure 3: Comparison of the convergence behaviors of GD, the adaptive gradient method (Adam), and the Gauss-Newton (GN) method. Left: Evolution of the function value over the iterations. Right: Condition numbers of the corresponding empirical kernels $\kappa(\Theta_k^{(M_k)})$.

where M_k^{-1} denotes the preconditioning matrix used by the optimizer. As before, we linearize the network output h_θ around θ_k at every training point and stack the resulting approximations into the vector $u(\theta)$. This yields the functional update

$$u(\theta_{k+1}) \approx u(\theta_k) - \alpha_k \Theta_k^{(M_k)} (u(\theta_k) - y),$$

with the *adaptive (or preconditioned) kernel*

$$\Theta_k^{(M_k)} := \frac{1}{m} J_k M_k^{-1} J_k^\top. \quad (15)$$

Thus, every adaptive and second-order method can be interpreted in function space as performing a kernel gradient descent with kernel $\Theta_k^{(M_k)}$. The eigenvalues of $\Theta_k^{(M_k)}$ therefore govern the contraction rates of the error for a given optimizer. In the special case of standard GD ($M_k = I$), we recover $\Theta_k^{(M_k)} = \Theta_k$, whose eigenvalues $\{\lambda_i\}$ produce per-iteration decay factors of the form $(1 - \alpha \lambda_i)^k$. Adaptive and second-order optimizers reshape these decay rates through the spectrum of $\Theta_k^{(M_k)}$. When M_k is chosen well, the resulting spectrum is significantly more balanced, narrowing the gap between the fastest- and slowest-contracting modes. This *spectral flattening* improves the effective conditioning of the optimization problem ($\kappa(\Theta_k^{(M_k)}) < \kappa(\Theta_k)$) and typically leads to faster and more stable training.

Numerical example #2: To illustrate the effect of preconditioning on optimization dynamics, we consider a nonlinear regression problem as specified in Section 2.1.2. Our goal is to learn the function $\sin(\pi x_1) + \cos(\pi x_2)$ from inputs (x_1, x_2) using a DNN. The model is trained using three different optimization methods. Specifically, we consider first-order GD, Adam as a representative of adaptive first-order methods, and a damped Gauss-Newton (GN) method as an approximate second-order optimization. GD corresponds to the choice $M_k = I$, while Adam employs a diagonal preconditioner M_k . The damped GN method

uses $M_k = J_k^\top J_k + \beta I$, where J_k denotes the Jacobian of the model outputs with respect to the parameters and $\beta > 0$ is a damping parameter.

Figure 3 reports the evolution of the objective function and the condition number of the associated kernel $\Theta_k^{(M_k)}$. As we can see, GD produces the most ill-conditioned kernel, which correlates with slow convergence. Adam’s diagonal preconditioning partially alleviates this issue, leading to slightly faster convergence. Notably, the GN method yields a substantially more balanced kernel spectrum, resulting in significantly smaller values of $\kappa(\Theta_k^{(M_k)})$ and the fastest empirical decrease of the objective function among the methods considered.

3.4 Impact of physical constraints on loss landscape and the NTK of PINNs

A key difficulty in training PINNs arises from the fact that the loss function does not act directly on the network output h_θ , but rather on its derivatives through the PDE operator. This has a profound influence on the geometry of the objective function landscape and on the conditioning of the NTK, and ultimately on the behaviour of employed optimization methods.

To understand this effect, let us consider the decomposition of h_θ in the Fourier components, i.e.,

$$h_\theta(x) = \sum_{\omega} \hat{h}(\omega) e^{i\omega x}$$

with $\hat{h}$ being the Fourier transform of h_θ . Let us now consider a Fourier mode $e^{i\omega x}$ present in $h_\theta(x)$. Applying p derivatives multiplies this mode by ω^p , so

$$\partial_x^p h_\theta \sim \omega^p, \quad J_k \sim \omega^p, \quad H(\theta) = \frac{1}{m} J_k^\top J_k \sim \omega^{2p}.$$

Thus, each Fourier mode of frequency ω approximately satisfies

$$H(\theta) q_\omega \approx \omega^{2p} q_\omega.$$

This produces a Hessian spectrum whose eigenvalues span

$$\lambda_{\min}(H(\theta_k)) \approx \omega_{\min}^{2p}, \quad \lambda_{\max}(H(\theta_k)) \approx \omega_{\max}^{2p},$$

where $\omega_{\min}$ and $\omega_{\max}$ are the smallest and largest frequencies that the network effectively represents. The condition number thus scales as $\kappa(H) \approx \left(\frac{\omega_{\max}}{\omega_{\min}}\right)^{2p}$. As a consequence, as the differential order p increases, the exponent $2p$ causes the ill-conditioning to worsen dramatically. This explains why higher-order PDEs exhibit far more severe NTK/Hessian pathologies than first- or second-order problems.

The same scaling directly affects the curvature of the PINN objective function. Low-frequency modes create shallow, nearly flat directions in the loss landscape, while high-frequency modes produce extremely steep ravines. This anisotropy makes the optimization problem stiff and highly challenging to solve.

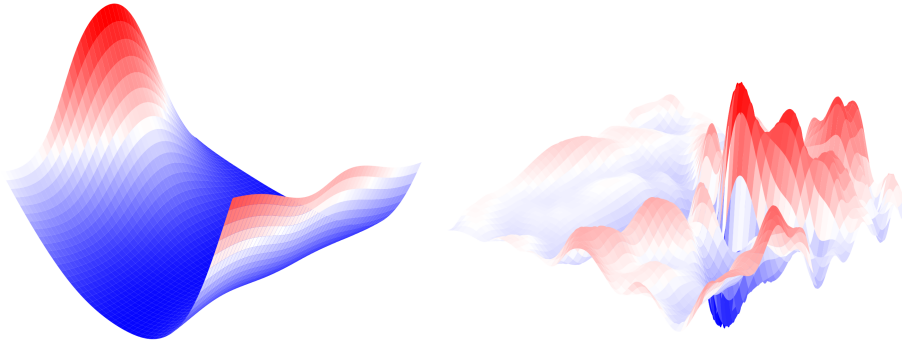


Figure 4: Two-dimensional projection [49] of the objective function landscape for the same DNN under two training regimes. Left: Data-driven nonlinear regression. Right: PINN.

First-order methods tend to move predominantly along low-frequency directions due to spectral bias, while second-order methods must cope with strong curvature anisotropy.

To illustrate these concepts, we consider two DNNs with identical architectures and parameter values to learn the solution of a one-dimensional Poisson equation. The first model is trained in a purely *data-driven* setting, formulated as a nonlinear regression problem using samples of the exact solution (Section 2.1.2). The second model is trained as a *PINN* (Section 2.1.3) by minimizing the PDE residual and the associated BC. Figure 4 compares the associated loss landscapes. Although both models approximate the same function u , the PINN loss landscape exhibits markedly more anisotropic curvature, while the data-driven objective remains comparatively smoother, albeit with several flat or low-curvature regions.

4 Stochastic gradient descent (SGD)

Stochastic Gradient Descent (SGD) is the most widely used optimization method in modern ML, and it plays an equally central role in data-driven SciML. Its popularity stems from its simplicity, scalability, and ability to train high-dimensional models on massive datasets. This widespread use is largely due to the limitations of classical GD, which, when applied to the finite-sum problem (2), requires evaluating the full gradient:

$$\nabla f(\theta) = \frac{1}{m} \sum_{i=1}^m \nabla f_i(\theta).$$

When the dataset size (m) is large, evaluating the full gradient quickly becomes computationally prohibitive. However, real-world datasets are highly

redundant, and computing every per-sample gradient at each iteration usually provides little additional benefit.

These considerations motivate the shift from classical deterministic optimization to *stochastic optimization*. The finite-sum structure of ML problems allows one to replace the full gradient with a cheaper approximation. The most famous method in this class is the stochastic gradient (SGD). The SGD method chooses a sample randomly at each iteration and utilizes just the corresponding gradient. Its iterations thus read as follows:

$$\theta_{k+1} = \theta_k - \alpha_k \nabla f_{i_k}(\theta_k), \quad (16)$$

where $\alpha_k > 0$ is a given step size. The symbol i_k denotes an index drawn uniformly at random from $\{1, \dots, m\}$ at each iteration.

Each iteration of the SGD method is thus very cheap, involving only the computation of the gradient $\nabla f_{i_k}(\theta_k)$, corresponding to one sample. This is a stochastic algorithm, meaning that the generated sequence is not uniquely determined from the starting point x_0 , but depends on the random sequence of indexes $\{i_k\}$. Note that each direction $-\nabla f_{i_k}(\theta_k)$ might not be one of descent for f_m (in the sense of yielding a negative directional derivative), but we can show that if it is a descent direction in expectation (the expected value of the scalar product with $\nabla f(\theta_k)$ is negative), then the sequence $\{\theta_k\}$ can be guided toward a minimizer of f_m . This comes from the fact that $\nabla f_{i_k}(\theta_k)$ is an unbiased estimator of $\nabla f(\theta_k)$, i.e.,

$$\mathbb{E}[\nabla f_{i_k}(\theta_k) | \theta_k] = \sum_{i=1}^m \frac{1}{m} \nabla f_i(\theta_k) = \nabla f(\theta_k).$$

The SGD method is summarized in Algorithm 1. Note that progress is in practice measured in epochs rather than iterations, where an epoch represents a complete pass over the whole dataset.

Algorithm 1 Stochastic Gradient (SGD) Method

- 1: **Given:** A dataset $\{x_i, y_i\}_{i=1}^m$, the objective functions $\{f_i\}_{i=1}^m$, an initial iterate $x_0 \in \mathbb{R}^n$
 - 2: **for** $k = 0, 1, 2, \dots$ **do**
 - 3: Randomly choose $i_k \in \{1, \dots, m\}$ ▷ Sample-index selection
 - 4: $p_k = -\nabla f_{i_k}(\theta_k)$ ▷ Search direction computation
 - 5: Choose $\alpha_k > 0$ ▷ Step size selection
 - 6: $\theta_{k+1} = \theta_k + \alpha_k p_k$ ▷ Iterate update
 - 7: **end for**
-

The SGD method often exhibits rapid progress in the early stages of training but may stagnate as training proceeds. This behavior is commonly attributed to the inherent noise in stochastic gradient estimates, which limits the attainable optimization accuracy. In many ML settings, this is not necessarily problematic, since ERM only approximates the expected risk, and excessive optimization of

the empirical objective may lead to overfitting, i.e., degraded performance on unseen data.

Mini-batch methods provide an intermediate approach between full-gradient methods, commonly referred to as batch gradient descent in this context, and SGD. Whereas SGD relies on a single sample at each iteration, mini-batch methods compute gradient estimates using a small subset $\mathcal{I}_k \subset \{1, \dots, m\}$ of the samples at each iteration, giving rise to the following update rule:

$$\theta_{k+1} = \theta_k - \frac{\alpha_k}{|\mathcal{I}_k|} \sum_{i \in \mathcal{I}_k} \nabla f_i(\theta_k).$$

This allows exploiting a degree of parallelism when computing mini-batch gradients, as each sample contribution can be evaluated independently. In addition, one often finds that, due to the reduced variance of the gradient estimates, the mini-batch methods are easier to tune in terms of choosing the step sizes.

In practice, selecting an appropriate stepsize is challenging, as its value directly affects the method's convergence. The stepsize α_k is often chosen heuristically. For example, a small constant value may be fixed to ensure convergence. Alternatively, a stepsize schedule can be employed to progressively decrease α_k . As we will see in the following section, mini-batch methods can converge with fixed step sizes, whereas SGD typically requires a decreasing step size.

4.1 Theoretical results

In this section, we present key convergence results for SGD applied to the finite-sum minimization problem (2). We outline one representative proof to illustrate the main proof techniques, while additional results and proofs can be found in [21]. Throughout, we denote by θ^* a solution of (2). Note, the discussed results remain valid if the stochastic gradient ∇f_{i_k} is replaced by any unbiased gradient estimator g_k satisfying $\mathbb{E}[g_k] = \nabla f(\theta_k)$.

The stated results mainly depend on the assumptions we make about the objective function. One quite common assumption is smoothness.

Definition 1. A function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is said to be smooth if there exists $L > 0$ such that, for all $\theta_1, \theta_2 \in \mathbb{R}^n$,

$$\|\nabla f(\theta_1) - \nabla f(\theta_2)\| \leq \|\theta_1 - \theta_2\|.$$

Assumption 1. Each function $f_i : \mathbb{R}^n \rightarrow \mathbb{R}$ in problem (2) is L_i -smooth. Denote $L_{\max} := \max_{i=1, \dots, n} L_i$.

Another important assumption, which, however, restricts the class of functions under study, is the convexity assumption.

Definition 2. A function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is said to be convex if for all $\theta_1, \theta_2 \in \mathbb{R}^n$, for all $t \in (0, 1)$

$$f(t\theta_1 + (1-t)\theta_2) \leq tf(\theta_1) + (1-t)f(\theta_2).$$

Definition 3. A function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is said to be strongly convex if there exists $\mu > 0$ such that for all $\theta_1, \theta_2 \in \mathbb{R}^n$

$$f(\theta_2) \geq f(\theta_1) + \nabla f(\theta_1)^T(\theta_2 - \theta_1) + \frac{\mu}{2} \|\theta_1 - \theta_2\|^2.$$

When needed, we will assume the following.

Assumption 2. Each function $f_i : \mathbb{R}^n \rightarrow \mathbb{R}$ in problem (2) is convex.

Convexity of the function usually makes the solution of the problem easier, since the landscape will be more benign (for instance, all the local minima are global minima for convex functions). With this assumption, we can obtain stronger convergence results. For instance, we will see that when f is convex, we can prove a convergence result on the sequence of functions (even on the sequence of iterates when f is strongly convex), while in the nonconvex case, we can just show a result on the norm of the gradient.

Another important factor that influences the convergence is the variance of the gradient estimate.

Assumption 3. Let

$$\sigma^2(\theta) := \mathbb{E}[\|\nabla f_i(\theta) - \nabla f(\theta)\|^2]$$

be the variance of the gradient estimate, and assume that this is bounded: $\sigma^2(\theta) \leq \sigma^2$ for all θ .

We will see that a non-zero variance will negatively impact the convergence of the optimization algorithms, which is why variance reduction techniques [6] may be used to obtain better stochastic directions and relax the assumptions on the step size, which also impacts the results. In this chapter, we consider both constant ($\alpha_k = \alpha$ for all k) and vanishing ($\alpha_k = \mathcal{O}(1/k)$) step sizes.

We will see that in all the results, the key constants that determine the convergence rates (smoothness $L_{\max}$, strong convexity μ , and the SGD variance) are directly related to the eigenvalues of the Hessian $H(\theta)$. In particular, we recall that, if f is μ strongly convex,

$$L_{\max} = \sup_{\theta} \lambda_{\max}(H(\theta)), \quad \mu = \inf_{\theta} \lambda_{\min}(H(\theta)), \quad \kappa(H(\theta)) = \frac{\lambda_{\max}(H(\theta))}{\lambda_{\min}(H(\theta))}.$$

Thus, the magnitudes of $\lambda_{\min}$ and $\lambda_{\max}$, as well as their ratio $\kappa(H(\theta))$, play a crucial role in interpreting the obtained theoretical convergence guarantees.

The convergence theorems show in particular how SGD deteriorates with increasing $\kappa(H(\theta))$.

4.1.1 Convergence for strongly convex and smooth functions

The first result applies to strongly convex functions, i.e., the ones with the most favorable landscape.

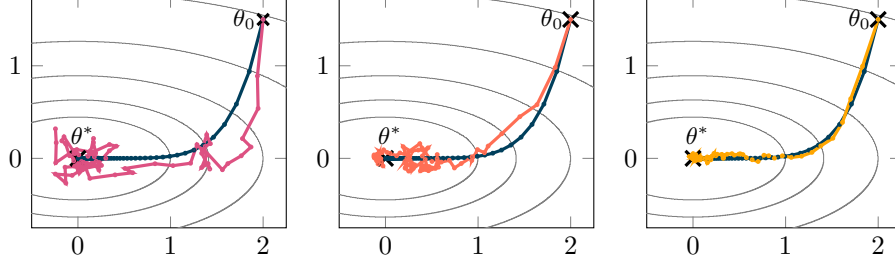


Figure 5: Trajectories of GD (blue) and SGD on a smooth quadratic function for different noise levels $\sigma \in \{\sigma_0, \sigma_0/2, \sigma_0/4\}$ (purple, orange, yellow). Higher noise magnitudes enlarge the region in which SGD oscillates around the minimizer θ^* , while lower noise confines the iterates closer to the GD path.

Theorem 1. *Let assumptions 1, 2 and 3 hold, and assume that f is μ -strongly convex. Assume that $\{\theta_k\}_{k \in \mathbb{N}}$ is the sequence generated by SGD (16) with a constant step size $0 < \alpha_k = \alpha < \frac{1}{2L_{\max}}$ for all k . Then, for $k \geq 1$, one has*

$$\mathbb{E}\|\theta_k - \theta^*\|^2 \leq (1 - \alpha\mu)^k \|\theta_0 - \theta^*\|^2 + \frac{2\alpha}{\mu} \sigma^2.$$

Proof. From the definition of the step (16) it holds

$$\begin{aligned} \|\theta_{k+1} - \theta^*\|^2 &= \|\theta_k - \alpha \nabla f_{i_k}(\theta_k) - \theta^*\|^2 \\ &= \|\theta_k - \theta^*\|^2 - 2\alpha \langle \nabla f_{i_k}(\theta_k), \theta_k - \theta^* \rangle + \alpha^2 \|\nabla f_{i_k}(\theta_k)\|^2. \end{aligned}$$

Taking the conditional expectation given θ_k yields

$$\mathbb{E}_k \|\theta_{k+1} - \theta^*\|^2 = \|\theta_k - \theta^*\|^2 - 2\alpha \langle \nabla f(\theta_k), \theta_k - \theta^* \rangle + \alpha^2 \mathbb{E}_k \|\nabla f_{i_k}(\theta_k)\|^2.$$

From the definition of strong convexity it follows

$$\mathbb{E}_k \|\theta_{k+1} - \theta^*\|^2 \leq (1 - \alpha\mu) \|\theta_k - \theta^*\|^2 - 2\alpha(f(\theta_k) - f(\theta^*)) + \alpha_k^2 \sigma^2.$$

Taking full expectation and using a variance transfer (see [21, lemma 4.20]) gives the main recursion:

$$\begin{aligned} \mathbb{E}\|\theta_{k+1} - \theta^*\|^2 &\leq (1 - \alpha_k\mu) \mathbb{E}\|\theta_k - \theta^*\|^2 + 2\alpha_k^2 \sigma^2 + 2\alpha(2\alpha L_{\max} - 1) \mathbb{E}(f(\theta_k) - f(\theta^*)) \\ &\leq (1 - \alpha_k\mu) \mathbb{E}\|\theta_k - \theta^*\|^2 + 2\alpha_k^2 \sigma^2, \end{aligned}$$

where we have used the assumption on the step size. Recursively applying the above and summing up the resulting geometric series gives

$$\begin{aligned} \mathbb{E}\|\theta_{k+1} - \theta^*\|^2 &\leq (1 - \alpha\mu)^k \|\theta_0 - \theta^*\|^2 + 2 \sum_{j=0}^{k-1} (1 - \alpha\mu)^j \alpha^2 \sigma^2 \\ &\leq (1 - \alpha\mu)^k \|\theta_0 - \theta^*\|^2 + \frac{2\alpha\sigma^2}{\mu}. \end{aligned}$$

□

This result tells us that SGD does not converge with a constant step size. If the variance of the gradient estimate was zero, the generated sequence would converge in expectation to the solution with a fast linear rate (i.e., the error from one iterate to the other would decrease exponentially). A gradient estimate with nonzero variance leads to the addition of a constant positive term on the right-hand side, which breaks the monotone decrease of the error. This means that the method will have a nice linear convergence, similar to the classical GD, whose rate depends on the learning rate α , until we reach a neighborhood of radius proportional to α and to the variance, see also Figure 5 for illustration. In addition, a large α leads to a fast initial decrease but to a larger steady-state error, while a small value leads to a slower decrease, but we can eventually come closer to the minimum.

Note that the eigenvalues of the Hessian also impact the convergence. A small $\mu \approx \lambda_{\min}(H(\theta_k))$ makes the linear rate $(1 - \alpha\mu)$ close to 1, while it simultaneously amplifies the variance term $(2\alpha/\mu)\sigma^2$. Thus, SGD cannot make progress in flat directions, where the curvature is very small ($\lambda_{\min} \approx 0$), and instead accumulates noise there.

4.1.2 Convergence for convex and smooth functions

In the following theorem we state an ergodic result for SG, applied to convex functions, with fixed step size.

Theorem 2. *Let assumptions 1, 2 and 3 be satisfied. Assume that $\{\theta_k\}_{k \in \mathbb{N}}$ is the sequence generated by SGD (16) with constant step size $\alpha_k = \alpha \leq \frac{1}{4L_{\max}}$, for all k . Define, for every $K \geq 1$,*

$$\bar{\theta}_K := \frac{1}{K} \sum_{k=1}^K \theta_k.$$

Then,

$$\mathbb{E}[f(\bar{\theta}_K) - \inf f] \leq \frac{\|\theta_0 - \theta^*\|^2}{\alpha K} + 2\alpha\sigma^2. \quad (17)$$

In particular, if for a fixed K we set $\alpha = \frac{\alpha_0}{\sqrt{K}}$ with $\alpha_0 \leq \frac{1}{4L_{\max}}$, the two terms are balanced, giving

$$\mathbb{E}[f(\bar{\theta}_K) - \inf f] = \frac{\|\theta_0 - \theta^*\|^2}{\alpha_0 \sqrt{K}} + \frac{2\alpha_0\sigma^2}{\sqrt{K}} = \mathcal{O}\left(\frac{1}{\sqrt{K}}\right).$$

We can see that the decrease is slower than in the strongly convex case (it is sublinear) and that the variance has the same effect as for strongly convex functions. In this case, the largest eigenvalue of the Hessian plays a role. Since the step size α is limited by $\alpha \leq \frac{1}{4L_{\max}}$, a large $\lambda_{\max}$ (and hence a large condition number) forces very small step sizes. Moreover, reducing α also increases the $1/(\alpha K)$ term in (17), delaying convergence.

Once again, convergence cannot be achieved with constant step sizes, but the effect of the variance can be counteracted by a vanishing step size, as shown in the following theorem.

Theorem 3. *Let assumptions 1, 2 and 3 be satisfied. Assume that $\{\theta_k\}_{k \in \mathbb{N}}$ is the sequence generated by SGD (16) with a vanishing step size $\alpha_k = \alpha_0/\sqrt{k+1}$ with $\alpha_0 \leq \frac{1}{4L_{\max}}$, and define*

$$\bar{\theta}_K := \frac{1}{\sum_{k=0}^{K-1} \alpha_k} \sum_{k=0}^{K-1} \alpha_k \theta_k.$$

Then,

$$\mathbb{E}[f(\bar{\theta}_K) - \inf f] \leq \frac{5\|\theta_0 - \theta^*\|^2}{4\alpha_0\sqrt{K}} + \sigma^2 \frac{5\alpha_0 \log(K+1)}{\sqrt{K}} = \mathcal{O}\left(\frac{\log(K+1)}{\sqrt{K}}\right).$$

4.1.3 Convergence for nonconvex and smooth functions

In this section, we just assume that the stochastic gradients are Lipschitz. Note that in this nonconvex setting, we cannot prove global optimality results. Nevertheless, we can still obtain bounds on the stationarity of the algorithm, stated in the following theorem.

Theorem 4. *Let Assumptions 1 and 3 be satisfied. Assume that $\{\theta_k\}_{k \in \mathbb{N}}$ is the sequence generated by SGD (16) with a constant step size $\alpha_k = \alpha = \sqrt{\frac{2}{L_f L_{\max} K}}$ for all k . Then, for all $K \geq 1$,*

$$\min_{k=0, \dots, K-1} \mathbb{E}\|\nabla f(\bar{\theta}_K)\|^2 \leq \frac{\sqrt{2L_f L_{\max}} \left(2(f(\theta_0) - \inf f) + \Delta_f^*\right)}{\sqrt{K}},$$

with $\Delta_f := \inf f - \frac{1}{m} \sum_{i=1}^m \inf f_i$. Consequently, for a given $\epsilon > 0$, if $K = \mathcal{O}(\epsilon^{-2})$ then

$$\min_{k=0, \dots, K-1} \mathbb{E}\|\nabla f(\bar{\theta}_K)\|^2 = \mathcal{O}(\epsilon).$$

This means that we need $\mathcal{O}(\epsilon^{-2})$ iterations to drive the norm of the gradient below the tolerance ϵ . Notice the constant hidden in the $\mathcal{O}(1/\sqrt{K})$ term depends on $L_{\max} = \sup_{\theta} \max_i |\lambda_i(H(\theta))|$. Thus, large curvature in some directions (large $L_{\max}$) makes the $\mathcal{O}(1/\sqrt{K})$ progress much slower in practice.

As we have seen, across all considered settings, the theoretical results imply that extremely small eigenvalues $\lambda_{\min}(H(\theta))$ lead to a slower contraction rate. Furthermore, large condition numbers $\kappa(H(\theta))$ enforce very small step sizes and the variance scales like $1/\lambda_{\min}(H(\theta))$, in turn amplifying noise as $\lambda_{\min}$ decreases, if $\lambda_{\min} < 1$. These observations provide an intuition for why first-order methods, such as (S)GD, struggle when training ill-conditioned problems, such as PINNs.

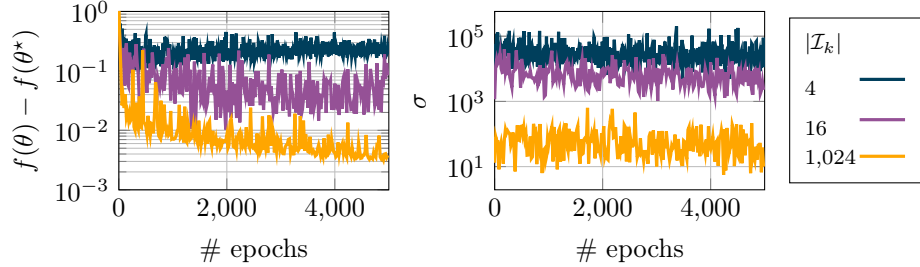


Figure 6: Effect of batch size on convergence of the mini-batch GD method with fixed $\alpha_k = 10^{-3}$. Left: Suboptimality $f(\theta_k) - f(\theta^*)$ over epochs for three batch sizes ($|\mathcal{I}| = \{4; 16; 1,024\}$). Right: Corresponding relative variance of subsampled gradient.

4.2 Numerical Example #3: SGD implementation

In this section, we demonstrate how the theoretical results discussed in Section 4.1 manifests in practice. To illustrate the behaviour of SGD, we consider the logistic regression problem introduced in Section 2.1 with a synthetic dataset. Figure 6 shows the effect of the mini-batch size on the behaviour of the SGD method. With a fixed learning rate and very small batches, SGD initially makes rapid progress but soon stagnates as the stochastic variance in the gradient estimates becomes dominant, preventing a consistent decrease in the loss. In contrast, larger batches reduce the sampling noise and produce search directions that more closely approximate the full gradient.

In practice, two standard mechanisms can be used to reduce the variance of stochastic search directions, whose effect is illustrated in Figure 7. The first approach is *learning rate scheduling*, where the step size α_k is incrementally reduced during training. In our experiments, we employ a *polynomial decay* schedule of the form

$$\alpha_k = \frac{\alpha_0}{(1 + \tau k)^{pw}},$$

with initial learning rate $\alpha_0 = 10^{-3}$, decay coefficient $\tau = 2 \cdot 10^{-2}$, and exponent $pw = 1$. Reducing the learning rate decreases the magnitude of stochastic perturbations and, as predicted theoretically, it enforces a gradual reduction of variance [73, 6]. Note that other schedules, such as step decay, exponential decay, or cosine annealing, are commonly used in modern practice [51].

The second approach is *batch-size scheduler*, which reduces variance not by shrinking α_k but by increasing the accuracy of each gradient estimate. In our experiment, the batch size evolves linearly from $|\mathcal{I}_0| = 16$ to $|\mathcal{I}_T| = 1,024$ over T epochs as

$$|\mathcal{I}_k| = |\mathcal{I}_0| + \frac{k}{T}(|\mathcal{I}_T| - |\mathcal{I}_0|), \quad k = 0, \dots, T.$$

The learning rate is kept constant at $\alpha_k = \alpha_0$, enabling progress in regimes

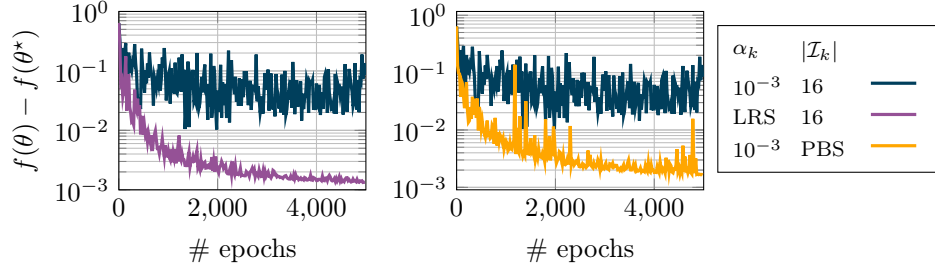


Figure 7: Example of reducing SGD noise by using learning rate scheduling (LRS) and progressive batch-size (PBS) increase. The SGD (blue) utilizes $|\mathcal{I}_k| = 16$ and $\alpha_k = 10^{-3}$. The SGD with LRS (purple) utilizes polynomial decay of α_k , while SGD with PBS progressively increases the batch size $|\mathcal{I}_k|$.

where small-batch SGD with a fixed step size would stagnate [81, 78].

Figure 7 demonstrates the performance of these two approaches, compared to SGD with $\alpha_k = 10^{-3}$ and $|\mathcal{I}_k| = 16$. As we can see, both learning rate decay and batch-size growth strategies effectively reduce the variance of the search direction, thereby restoring convergence. However, they differ subtly in practice: learning rate decay improves stability but slows down the method permanently, whereas batch-size growth preserves the step magnitude and reduces noise without sacrificing asymptotic speed, but progressively increases the iteration cost.

5 Importance of sampling strategies for the training of PINNs

The performance of stochastic first-order optimization for PINNs depends crucially on *how the collocation points are sampled*. Unlike classical ML, where data points are fixed and drawn from a true distribution, in PINNs, the user explicitly chooses the sampling distribution. This choice directly shapes the optimization landscape, the NTK/Hessian spectrum, and the overall convergence behavior. In this section, we explain why sampling matters, how mini-batch SGD behaves differently in PINNs compared to standard ML, and why naive uniform sampling often leads to poor or unstable convergence.

5.1 Choice of collocation points

As we have seen in Section 2.1.3, PINNs approximate the PDE residual and BC conditions at a finite set of collocation points. Thus, the optimization problem depends directly on *where* the PDE/BC are sampled. Recall that, from Section 3.1, the empirical NTK is given as $\Theta_k = \frac{1}{m} \sum_{j=1}^m \nabla_{\theta} h_{\theta_k}(x_j) \nabla_{\theta} h_{\theta_k}(x_j)^{\top}$. Thus, each collocation point contributes one rank-one matrix to the definition of the kernel. If important regions of the domain are undersampled (e.g., boundary

layers, sharp interfaces), then the corresponding Jacobian directions are either missing or severely underrepresented in the sum. This manifests as very small or even vanishing eigenvalues of Θ_k , meaning that the NTK becomes nearly singular. Conversely, oversampling smooth regions causes certain directions to dominate and inflate the largest eigenvalues.

Both effects increase the condition number $\kappa(\Theta_k)$, leading to small and highly anisotropic gradient updates. Hence, the quality of the sampling distribution directly determines the spectral properties of the NTK/Hessian spectrum, which in turn determines the difficulty of the optimization problem, as we have seen in Section 4. As a result, even with a fixed neural architecture, the sampling strategy alone can dictate whether optimization methods converge rapidly or not. Hence, the design of the collocation set is not merely a modelling choice. It is also an *optimization parameter* that can dramatically affect training, especially for problems with nonuniform PDE stiffness, such as those occurring in boundary-layer flows, or advection-dominated regimes.

5.2 Adaptive choice of collocation points

A practical way to mitigate NTK/Hessian ill-conditioning caused by poor sampling is to adaptively relocate or enrich collocation points in regions identified as problematic. This idea is closely related to adaptive mesh refinement in classical numerical PDE methods [88].

Let $\mathcal{P}^{(t)}$ denote the sampling distribution at refinement stage t , with density $p^{(t)}(x)$ supported on Ω , and let

$$\mathcal{D}_\Omega^{(t)} = \{x_j^{(\Omega,t)}\}_{j=1}^{m_\Omega^{(t)}}$$

be the corresponding collocation set. Adaptive resampling can be performed every T iterations, after which training resumes from the current parameters θ , creating a feedback loop between optimization and sampling.

Given an indicator $\eta^{(t)}(x)$ measuring for example the PDE residual [95, 59], or uncertainty [99, 64], the sampling density is updated according to

$$p^{(t+1)}(x) = \frac{(\eta^{(t)}(x))^\beta}{\int_\Omega (\eta^{(t)}(\tilde{x}))^\beta d\tilde{x}}, \quad \beta > 0.$$

Here, larger values of β concentrate points in regions where the indicator $\eta^{(t)}$ is large, while smaller values promote broader exploration.

This adaptive sampling strategy can be interpreted as importance sampling for the residual empirical risk

$$\hat{R}_\Omega(\theta) = \frac{1}{m_\Omega} \sum_{j=1}^{m_\Omega} \frac{1}{p^{(t)}(x_j^{(\Omega)})} |\mathcal{N}[h_\theta](x_j^{(\Omega)}) - q(x_j^{(\Omega)})|^2,$$

which yields unbiased gradient estimates under nonuniform sampling [66].

Condition number From a spectral viewpoint, adaptive sampling improves the conditioning of the empirical NTK/Hessian. Recall Section 3.1, the NTK is defined as a sum of positive semidefinite (PSD) rank-one terms, i.e., $\Theta_m = \frac{1}{m} \sum_{j=1}^m J_j J_j^\top$, where m denotes the number of utilized collocation points.

Adding a new collocation point yields

$$\Theta_{m+1} = \frac{m}{m+1} \Theta_m + \frac{1}{m+1} J_{m+1} J_{m+1}^\top, \quad J_{m+1} J_{m+1}^\top \succeq 0.$$

Now, let $A := \frac{m}{m+1} \Theta_m$, and $B := \frac{1}{m+1} J_{m+1} J_{m+1}^\top$, so that $\Theta_{m+1} = A + B$ with $B \succeq 0$. By Weyl's inequality for Hermitian matrices [29, Theorem 4.3.1], it follows that

$$\lambda_i(A + B) \geq \lambda_i(A), \quad i = 1, \dots, n.$$

Therefore,

$$\lambda_i(\Theta_{m+1}) \geq \lambda_i\left(\frac{m}{m+1} \Theta_m\right) = \frac{m}{m+1} \lambda_i(\Theta_m) \geq 0, \quad i = 1, \dots, n.$$

Hence, in practice, when the new Jacobian J_{m+1} introduces a direction not yet represented in $\{J(x_j)\}_{j=1}^m$, the smallest eigenvalue $\lambda_{\min}(\Theta_m)$ increases sharply, while $\lambda_{\max}(\Theta_m)$ is nearly unaffected. This expands the lower end of the NTK spectrum, improving the condition number $\kappa(\Theta_m)$. Geometrically, the quadratic form $\theta^\top \Theta_m \theta$ determines the level sets of the loss landscape. Improving $\kappa(\Theta_m)$ corresponds to making these level sets more isotropic, which in turn permits larger stable step sizes and faster convergence of the GD method.

5.2.1 Example: Adaptive Sampling for 1D Poisson problem

To illustrate how adaptive sampling can improve conditioning, we consider the one-dimensional Poisson problem on $\Omega = (0, 1)$, given as $-u''(x) = f(x)$, $u(0) = u(1) = 0$, with a localized forcing term

$$f(x) = 10 e^{-100(x-0.9)^2},$$

which induces sharp gradients in the solution near $x \approx 0.9$.

We approximate the solution by a simple two-parameter surrogate,

$$h(x; \theta) = \theta_1 \phi_1(x) + \theta_2 \phi_2(x), \quad \phi_1(x) = x(1-x), \quad \phi_2(x) = x^2(1-x)^2.$$

The residual and its Jacobian with respect to the parameters θ are given by

$$r(x; \theta) = -h''(x; \theta) - f(x), \quad J(x) = \nabla_\theta r(x; \theta) = \begin{bmatrix} -\phi_1''(x) \\ -\phi_2''(x) \end{bmatrix}.$$

The NTK is defined as $\Theta_m = \frac{1}{m} \sum_{j=1}^m J(x_j) J(x_j)^\top$, where m denotes the number of utilized collocation points.

Uniform sampling Let the collocation points be uniformly spaced as

$$x_j \in \{0.1, 0.3, 0.5, 0.7, 0.9\}.$$

Most of these points lie in regions where $f(x)$ and $r(x)$ are small, resulting in Jacobians that are nearly collinear. Consequently, the resulting NTK, $\Theta_5 = \frac{1}{5} \sum_{j=1}^5 J(x_j)J(x_j)^\top$, has eigenvalues $\lambda_{\max} \approx 4.1$ and $\lambda_{\min} \approx 0.02$, yielding a condition number $\kappa(\Theta_5) \approx 200$.

Adaptive refinement Now, let us suppose that a residual-based sampling identifies the high-error region near $x \approx 0.9$ and enriches the collocation set to

$$x_j \in \{0.1, 0.3, 0.5, 0.7, 0.85, 0.9, 0.95\},$$

as illustrated in Figure 8. The additional points contribute Jacobian directions aligned with the localized feature in $f(x)$. The refined NTK, $\Theta_7 = \frac{1}{7} \sum_{j=1}^7 J(x_j)J(x_j)^\top$, exhibits $\lambda_{\max} \approx 4.0$ and $\lambda_{\min} \approx 0.30$, in turn reducing the condition number to $\kappa(\Theta_7) \approx 13$. Note that the smallest eigenvalue increases significantly, while the largest remains nearly unchanged.

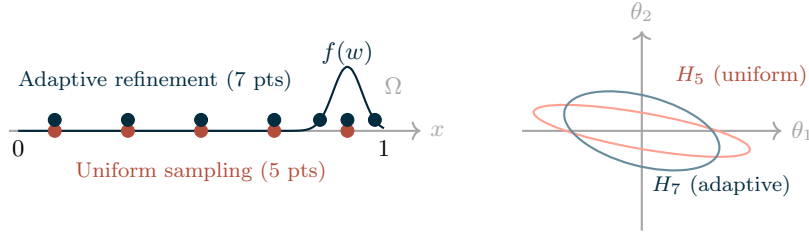


Figure 8: (*Left*) Domain $\Omega = (0,1)$ with localized forcing near $x \approx 0.9$. Uniform sampling is depicted in orange, while adaptive refinement is depicted in blue. (*Right*) Level sets of $\theta^\top H_m \theta = 1$ become more isotropic after refinement, reflecting improved conditioning.

5.3 Mini-batch sampling

As discussed in Section 4, SGD relies on stochastic gradient estimates computed using mini-batches. For classical ML tasks with i.i.d. data, such mini-batch gradients provide an unbiased estimator of the full gradient with controlled variance. In PINNs, however, selecting informative and stable mini-batches is substantially more delicate, for several reasons:

1. *Non-i.i.d. sampling:* When collocation points are generated using structured sampling strategies, such as quasi-Monte Carlo, or adaptive schemes, the samples are no longer independent. As a result, classical variance-based analyses of SGD do not apply directly. Instead, the accuracy of

mini-batch gradient estimates is governed by discrepancy or dependence properties of the point set rather than by variance reduction. While such sampling strategies often yield smoother and more uniform gradient estimates in practice, they may introduce bias unless appropriate randomization is employed, e.g., digital shifts or scrambling.

2. *Spatially correlated and heterogeneous sampling:* The PDE residual is spatially correlated and typically highly nonuniform across the domain. Mini-batches that contain points in steep-gradient regions generate disproportionately large updates, while batches drawn from smoother regions produce much smaller ones. This heterogeneity leads to inconsistent optimization steps and can induce oscillations or even divergence.
3. *Highly unbalanced variance:* For a squared-residual objective, the mini-batch gradient estimator can be written as

$$g_{\mathcal{I}_k}(\theta) = \frac{1}{|\mathcal{I}_k|} \sum_{x_j \in \mathcal{I}_k} \nabla_{\theta} r_{\theta}(x_j) r_{\theta}(x_j), \quad \phi_{\theta}(x) := \nabla_{\theta} r_{\theta}(x) r_{\theta}(x).$$

Under i.i.d. Monte Carlo sampling, the variance scales as

$$\text{Var}[g_{\mathcal{I}_k}(\theta)] = \frac{1}{|\mathcal{I}_k|} \text{Var}[\phi_{\theta}(x)], \quad \mathbb{E}\|\phi_{\theta}(x)\|^2 = \mathbb{E}[r_{\theta}(x)^2 \|\nabla_{\theta} r_{\theta}(x)\|^2].$$

In PDE-constrained losses, $\nabla_{\theta} r_{\theta}$ involves derivatives of order p of h_{θ} and may therefore grow rapidly with the frequency content of the solution, recall Section 3.4. As a result, higher-order PDEs or solutions with sharp layers can produce large batch-to-batch fluctuations, destabilizing early training and slowing convergence.

4. *Interaction with NTK conditioning:* Even if the variance of the stochastic gradient is moderate, the mini-batch update is still filtered through the NTK Θ_k . From (14), each update is projected onto the eigenbasis of the NTK. Directions corresponding to small NTK eigenvalues are thus effectively frozen:

$$(I - \alpha \Theta_k) q_i = (1 - \alpha \lambda_i) q_i \approx q_i \quad \text{for } \lambda_i \ll 1,$$

so that SGD makes little progress in these components. However, stochastic gradient noise still projects onto these poorly conditioned subspaces, where the curvature is nearly zero, causing noise accumulation and spurious oscillations. As a consequence, optimization may appear to stagnate despite rapid progress in better-conditioned directions.

To avoid these difficulties, practitioners frequently bypass mini-batching altogether and instead use the *entire* set of collocation points during training. As a result, PINNs are often trained in a *deterministic* or *very large-batch* regime.

5.3.1 Practical mini-batching strategies

Despite the aforementioned challenges, several works report that mini-batch training can improve generalization in PINNs [91, 45, 33]. However, the SciML literature currently provides no principled guidelines for constructing mini-batches tailored to PINNs. Existing strategies typically emerge from collocation sampling heuristics or efforts to mitigate ill-conditioning rather than from an explicit analysis of mini-batch SGD. Below, we summarize some approaches that implicitly guide mini-batch design.

- *Importance sampling:* In classical ML, importance sampling selects data points with probability proportional to the magnitude of their gradient, thereby reducing the variance of the subsampled gradient. For PINNs, an analogous strategy is to sample collocation points according to [95, 66]:

$$p(x) \propto |r(x; \theta)|^\beta, \quad \beta > 0.$$

This biases computation toward regions where the residual is currently largest, effectively prioritizing harder parts of the PDE.

- *Stratified sampling across interior and boundary:* Several studies observe that mini-batches containing too few boundary points lead to unstable training [13, 53]. A practical remedy is to partition the dataset as

$$\mathcal{D} = \mathcal{D}_\Omega \cup \mathcal{D}_{\Gamma_D} \cup \mathcal{D}_{\Gamma_N},$$

and to construct each mini-batch by sampling a prescribed fraction of points from each subset. Such stratified mini-batches balance the gradient contributions from interior and boundary terms.

- *Spatially informed batching:* Because the PDE residual is spatially correlated, mini-batches composed of nearby points tend to yield highly correlated gradients [45, 90]. To counteract this, one may enforce spatial diversity within a batch, for example by sampling from a coarse grid, selecting points from disjoint spatial partitions, or ensuring coverage of distant subregions of Ω .

6 Accelerated and adaptive stochastic gradient methods

The main limitation of standard first-order methods is their slow convergence, a behaviour often attributed to the difficulty of selecting an appropriate step size. This issue becomes even more pronounced in large-scale SciML, where physical constraints, large models, noisy gradients, and heterogeneous data make choosing a suitable step size both challenging and essential for stable and efficient training. To address these challenges, a wide range of accelerated and adaptive variants of SGD have been developed. In this section, we present the most widely used.

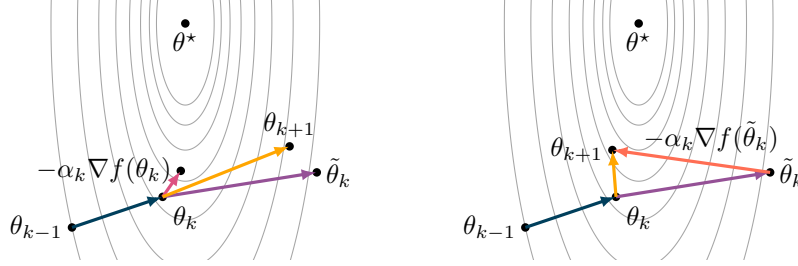


Figure 9: Sketch of heavy-ball momentum (left) and NAG (right) iteration.

6.1 Momentum

Its purpose is to mitigate a key limitation of GD, namely that the parameter update at iteration k depends only on local information, such as the learning rate and the gradient, and does not exploit the history of previous updates. Incorporating past information can provide valuable insight into the objective function landscape along the optimization trajectory and can significantly improve convergence behavior.

Momentum techniques keep memory of past gradients and accumulate them. In order to take into account a notion of time, they use an exponentially weighted average to give more importance to recent gradients. The update at iteration k then becomes

$$p_k = \beta p_{k-1} + (1 - \beta) g_k,$$

where p_k is the new search direction. The momentum parameter $\beta \in (0, 1)$ controls the weighting between past and current gradients, and g_k is the gradient approximation at iteration k (which may be the full gradient, a mini-batch gradient, or a stochastic gradient).

The term $(1 - \beta)$ is often replaced with the learning rate, and a dynamically adjusted parameter β_k may be used instead of a fixed β to scale p_{k-1} , so the general update for GD with momentum is given as

$$\theta_{k+1} = \theta_k - \alpha_k g_k + \beta_k (\theta_k - \theta_{k-1}), \quad (18)$$

where the scalar sequence $\{\alpha_k\}$ is either predetermined or set dynamically.

Heavy ball If $\alpha_k = \alpha$ and $\beta_k = \beta$ for some constants $\alpha > 0$ and $\beta \in (0, 1)$ and for all $k \in \mathbb{N}$, the update (18) is referred to as the *heavy ball method*. An alternative interpretation of this method is obtained by expanding the update:

$$\theta_{k+1} = \theta_k - \alpha \sum_{j=1}^k \beta^{k-j} g_j.$$

Thus, each step can be viewed as an exponentially weighted average of past gradients. Larger values of β place greater emphasis on gradients from earlier iterations. In practice, β is typically chosen close to 0.9.

Nesterov accelerated gradient (NAG) Another widely used variant of momentum is the Nesterov accelerated gradient (NAG). The core idea behind NAG is that if the current parameter vector is θ_k , then the momentum term alone (i.e., ignoring the term with the gradient) is about to nudge the parameter vector by $\beta_k(\theta_k - \theta_{k-1})$. Therefore, it makes sense to compute the gradient at $\theta_k + \beta_k(\theta_k - \theta_{k-1})$ instead of at the old position θ_k .

Written as a two-step procedure, the NAG update is given as

$$\tilde{\theta}_k = \theta_k + \beta_k(\theta_k - \theta_{k-1}) \quad \text{and} \quad \theta_{k+1} = \tilde{\theta}_k - \alpha_k \nabla f(\tilde{\theta}_k),$$

which can be reformulated as

$$\theta_{k+1} = \theta_k - \alpha_k \nabla f(\theta_k + \beta_k(\theta_k - \theta_{k-1})) + \beta_k(\theta_k - \theta_{k-1}). \quad (19)$$

The main difference with classical momentum is thus the order of computation (momentum step versus gradient step), see also Figure 9.

6.2 Adaptive gradient algorithms: AdaGrad, Adam

In order to overcome the limitation of SGD concerning the learning rate choice, adaptive optimization algorithms that adjust the learning rate for each parameter individually based on past gradients have been proposed. The most notable examples are AdaGrad and Adam.

Adaptive Gradient(AdaGrad) [18] AdaGrad updates parameters θ at iteration k in component-wise manner, i.e.,

$$\theta_{k+1}(i) = \theta_k(i) - \frac{\alpha}{\sqrt{G_k(i) + \epsilon}} g_k(i), \quad (20)$$

where the symbol (i) denotes i -th component of the vector and g_k stands for the gradient approximation at iteration k . The vector G_k contains the sum of squares of past gradients for each parameter, i.e., $G_k(i) = \sum_{j=1}^k g_j(i)^2$. The symbol α denotes the global learning rate, and ϵ is a small constant to avoid division by zero.

AdaGrad gives larger updates to infrequent parameters and smaller updates to frequent ones, which can be particularly useful for sparse data. However, the accumulated squared gradients in G_k increase indefinitely and can cause the effective learning rate to decrease excessively over time.

Adaptive Moment Estimation (Adam) [39] Adam improves upon AdaGrad by combining adaptive learning rates with momentum. This is achieved by maintaining exponentially decaying averages of past gradients (first moment) and squared gradients (second moment):

$$\begin{aligned} m_k(i) &= \beta_1 m_{k-1}(i) + (1 - \beta_1) g_k(i), \\ v_k(i) &= \beta_2 v_{k-1}(i) + (1 - \beta_2) (g_k(i))^2, \end{aligned}$$

which are then bias-corrected as

$$\begin{aligned}\hat{m}_k(i) &= \frac{m_k(i)}{1 - \beta_1^k}, \\ \hat{v}_k(i) &= \frac{v_k(i)}{1 - \beta_2^k},\end{aligned}$$

where β_1 and β_2 are decay rates for the moving averages.

The moments $\hat{m}_k$ and $\hat{v}_k$ are then used to update the current iterate in the following, component-wise, manner:

$$\theta_{k+1}(i) = \theta_k(i) - \alpha \frac{\hat{m}_k(i)}{\sqrt{\hat{v}_k(i)} + \epsilon}. \quad (21)$$

Adam maintains an exponentially decaying moving average of squared gradients, which remains bounded and thus avoids the vanishing learning-rate issue inherent to AdaGrad’s cumulative accumulation of squared gradients. Moreover, it also avoids giving more importance to the most recent gradients. As a result, Adam combines the benefits of AdaGrad’s per-parameter learning rates with momentum’s acceleration, making it robust to noisy or sparse gradients. It is nowadays implemented in all the ML libraries, and it is one of the most widely used optimizers.

Numerical example # 4: We now empirically compare the performance of SGD, NAG, AdaGrad, and Adam on the same logistic-regression problem as considered in Section 4.2. All methods are run in the full-batch regime ($|\mathcal{I}_k| = 6000$), ensuring that differences in convergence arise solely from the update rule rather than stochastic sampling effects. Step sizes are chosen to guarantee stable behaviour: SGD and NAG use a small step size $\alpha_k = 10^{-4}$, whereas AdaGrad and Adam employ significantly larger values $\alpha_k = 10^{-2}$.

Figure 10 shows that standard SGD exhibits the slowest convergence. NAG and AdaGrad achieve faster progress during the initial phase, but both ultimately remain limited by the ill-conditioning. Adam, by contrast, displays the fastest overall convergence, illustrating how adaptive preconditioning can stabilise training while allowing the use of larger constant learning rates.

Finally, we emphasise that optimal step size choices and the relative performance of these optimisers are problem-dependent and must be tuned in practice. Moreover, in deep-learning applications, methods that progress slowly or conservatively (such as SGD) may explore the landscape more thoroughly and often yield solutions with better generalisation performance. As a result, identifying the most suitable optimization method and tuning it appropriately continues to be one of the key practical challenges in ML workflows.

7 Beyond first-order optimization

While first-order methods such as SGD and Adam dominate modern ML, they often struggle in SciML settings, motivating the use of curvature-aware ap-

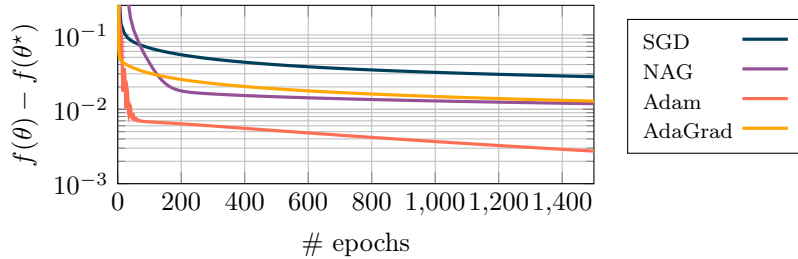


Figure 10: Example of performance of SGD and adaptive first-order methods on the logistic regression model problem described in Section 4.2.

proaches. Second-order methods, such as the Newton method, can converge much faster near a local minimizer, which is particularly beneficial in SciML, where the loss landscape is often stiff and highly anisotropic due to underlying differential equations. However, their high computational cost makes them impractical at scale, leading to the development of scalable alternatives such as Hessian-free inexact Newton, subsampled Newton, or quasi-Newton methods.

7.1 Hessian-free inexact Newton-Krylov methods

Hessian-free inexact Newton-Krylov methods [42] provide a scalable means of incorporating curvature information without explicitly forming or storing the Hessian. Rather than solving the Newton system

$$H(\theta_k)p_k = -\nabla f(\theta_k)$$

accurately, these methods compute an approximate Newton step using an iterative Krylov solver, e.g., the conjugate gradient method. Provided that the linear systems are solved to a tolerance that is progressively tightened as the iterations proceed, inexact Newton methods can achieve superlinear convergence [15].

A key advantage of this approach is that Krylov solvers require only Hessian-vector products. This is particularly attractive in SciML, where Hessian-vector products can often be computed efficiently using automatic differentiation or adjoint techniques. Although each Hessian-vector product is typically comparable in cost to a gradient evaluation, the improved convergence behavior can often compensate for the additional per-iteration cost. Such ideas have recently been employed for regression problems in SciML [97], demonstrating higher model accuracy or lower computational costs, compared to adaptive first-order methods.

7.2 Subsampled Hessian-free inexact Newton methods

Analogous to mini-batch gradient methods, the computational cost of Hessian-free inexact Newton-Krylov methods can be further reduced by employing sub-

sampled Hessian [74]. In this case, the step is given as

$$H_{\mathcal{I}_k^H}(\theta_k)p_k = -\nabla f_{\mathcal{I}_k}(\theta_k),$$

where we used index sets $\mathcal{I}_k \subset \{1, \dots, m\}$ and $\mathcal{I}_k^H \subset \{1, \dots, m\}$ to obtain the stochastic gradient and stochastic Hessian estimates as

$$\nabla f_{\mathcal{I}_k}(\theta_k) = \frac{1}{|\mathcal{I}_k|} \sum_{i \in \mathcal{I}_k} \nabla f_i(\theta_k), \quad H_{\mathcal{I}_k^H}(\theta_k) = \frac{1}{|\mathcal{I}_k^H|} \sum_{i \in \mathcal{I}_k^H} \nabla^2 f_i(\theta_k).$$

Here, we assumed that $\mathcal{I}_k^H$ is uncorrelated with $\mathcal{I}_k$ and $|\mathcal{I}_k^H| < |\mathcal{I}_k|$, since the iteration is more tolerant to noise in the Hessian estimate than it is to noise in the gradient estimate.

If one chooses the subsample size $|\mathcal{I}_k^H|$ small enough, then the cost of each product involving the Hessian approximation can be reduced significantly, thus reducing the cost of each Krylov iteration. Algorithm 2 summarizes the subsampled Newton method. The choice of the step size in this context is often done via line-search strategies [67, ch.3], adaptive techniques that ensure convergence of the method for any starting point.

Similar to stochastic first-order methods, subsampled Newton methods are not particularly well-suited for physics-informed problems due to the global coupling induced by the PDE constraints. However, they may be well suited for data-driven SciML problems, where the loss decomposes over independent training samples, e.g., operator learning, DNN surrogate models trained on simulation data, or latent-space representations learned from large collections of solution snapshots. In such cases, the Hessian admits a natural sample-wise decomposition, making subsampled Newton methods a potentially effective yet largely unexplored alternative to (adaptive) first-order optimization.

Algorithm 2 Subsampled Hessian-Free Inexact Newton Method

- 1: **Given:** A dataset $\{x_i, y_i\}_{i=1}^m$, an objective function $f : \mathbb{R}^n \rightarrow \mathbb{R}$, an initial iterate $\theta_0 \in \mathbb{R}^n$
- 2: **for** $k = 0, 1, 2, \dots$ **do**
- 3: Choose $\mathcal{I}_k^H$ and construct $H_{\mathcal{I}_k^H}(\theta_k)$ ▷ Hessian approximation
- 4: Choose $\mathcal{I}_k$ and $\nabla f_{\mathcal{I}_k}(\theta_k)$ ▷ Gradient approximation
- 5: Obtain p_k by approximately solving ▷ Inexact step computation

$$H_{\mathcal{I}_k^H}(\theta_k)p_k = -\nabla f_{\mathcal{I}_k}(\theta_k)$$

- 6: Choose α_k ▷ Step size selection
 - 7: Set $\theta_{k+1} = \theta_k + \alpha_k p_k$ ▷ Iterate update
 - 8: **end for**
-

7.3 Quasi-Newton methods

Quasi-Newton methods use the same update rule as Newton's method, but instead of the exact Hessian $H(\theta_k)$, they employ an approximation B_k . The

update is then given as $\theta_{k+1} = \theta_k + \alpha_k p_k$, where p_k is such that $B_k p_k = -\nabla f(\theta_k)$. The most well-known quasi-Newton method is perhaps BFGS [67, 50], which allows for directly approximating the inverse of the Hessian $\tilde{B}_k$, so that the iteration update further simplifies by avoiding the need to solve a linear system. Thus, the search direction is obtained as

$$p_k = \tilde{B}_k \nabla f(\theta_k).$$

At iteration k , the inverse Hessian approximation $\tilde{B}_k$ is constructed recursively, starting from an initial approximation $\tilde{B}_0$, according to

$$\tilde{B}_{k+1} = (I - \varrho_k s_k y_k^T) \tilde{B}_k (I - \varrho_k y_k s_k^T) + \varrho_k s_k s_k^T, \quad \varrho_k = \frac{1}{y_k^T s_k}, \quad (22)$$

where

$$s_k := \theta_{k+1} - \theta_k, \quad y_k := \nabla f(\theta_{k+1}) - \nabla f(\theta_k).$$

If $\tilde{B}_k$ is symmetric positive definite and the curvature condition $y_k^T s_k > 0$ holds, then $\tilde{B}_{k+1}$ is also symmetric positive definite.

Algorithm 3 L-BFGS two-loop recursion

- 1: **Given:** Current gradient $\nabla f(\theta_k)$, pairs $\{s_i, y_i\}_{i=1}^S$, $\varrho_i = 1/(y_i^T s_i)$, initial approximation of the inverse of the Hessian $\tilde{B}_k^0$
 - 2: Set $q = \nabla f(\theta_k)$
 - 3: **for** $i = k-1, k-2, \dots, k-S$ **do**
 - 4: Compute $\alpha_i = \varrho_i s_i^T q$
 - 5: Update $q = q - \alpha y_i$
 - 6: **end for**
 - 7: Set $r = \tilde{B}_k^0 q$
 - 8: **for** $i = k-S, k-S+1, \dots, k-1$ **do**
 - 9: Compute $\beta = \varrho_i y_i^T r$
 - 10: Update $r = r + s_i(\alpha_i - \beta)$
 - 11: **end for**
 - 12: **return** r
-

L-BFGS For large-scale problems, storing the full matrix $\tilde{B}_k$ is infeasible. This motivates the use of a limited-memory variant, termed L-BFGS, which stores only S most recent secant pairs (s_k, y_k) instead of the full $\tilde{B}_k$. These pairs suffice to approximate the application of $\tilde{B}_k$ to $\nabla f(\theta_k)$ in order to compute the search direction p_k , in turn reducing storage requirements from $O(n^2)$ to $O(nS)$.

To derive an efficient procedure for applying $\tilde{B}_k$ to $\nabla f(\theta_k)$, we rewrite the BFGS inverse update (22) in the compact form as

$$\tilde{B}_{k+1} = V_k^T \tilde{B}_k V_k + \varrho_k s_k s_k^T, \quad V_k := I - \varrho_k y_k s_k^T. \quad (23)$$

As a consequence, the product $\tilde{B}_k \nabla f(\theta_k)$ can be computed using only inner products and vector additions involving the stored secant pairs $\{(s_i, y_i)\}_{i=k-S}^{k-1}$. Thus, by repeatedly applying (23), we obtain

$$\begin{aligned}\tilde{B}_k &= (V_{k-1}^T \cdots V_{k-S}^T) \tilde{B}_k^0 (V_{k-S} \cdots V_{k-1}) \\ &\quad + \varrho_{k-S} (V_{k-1}^T \cdots V_{k-S+1}^T) s_{k-S} s_{k-S}^T (V_{k-S+1} \cdots V_{k-1}) \\ &\quad + \varrho_{k-S+1} (V_{k-1}^T \cdots V_{k-S+2}^T) s_{k-S+1} s_{k-S+1}^T (V_{k-S+2} \cdots V_{k-1}) \\ &\quad + \cdots + \varrho_{k-1} s_{k-1} s_{k-1}^T.\end{aligned}$$

This expression leads to a recursive scheme for computing $\tilde{B}_k \nabla f(\theta_k)$ efficiently, namely the two-loop recursion summarized in Algorithm 3. Assuming $\tilde{B}_k^0$ is diagonal, the two-loop recursion requires approximately $4Sn + n$ multiplications. A common choice for $\tilde{B}_k^0$ is a scaled identity $\tilde{B}_k^0 = \tau_k I$, with $\tau_k > 0$.

Algorithm 4 outlines the L-BFGS procedure. Note that when using L-BFGS, full gradients are typically required, as the method is sensitive to noise in gradient estimates. While noise-robust variants of BFGS and L-BFGS have been proposed [79, 31], this is not a significant limitation for physics-based models, which are usually trained in deterministic settings due to global coupling induced by PDE constraints. Consequently, L-BFGS has emerged as one of the most widely used optimizers for PINNs, see a comprehensive benchmark of quasi-Newton methods for PINNs provided in [40].

Algorithm 4 L-BFGS

- 1: **Given:** A dataset $\{x_i, y_i\}_{i=1}^m$, an objective function $f : \mathbb{R}^n \rightarrow \mathbb{R}$, an initial iterate $\theta_0 \in \mathbb{R}^n$, a memory size $S > 0$
 - 2: **for** $k = 0, 1, 2, \dots$ **do**
 - 3: Choose $\tilde{B}_k^0$ ▷ Initial inverse Hessian approximation
 - 4: Compute $p_k = -\tilde{B}_k \nabla f(\theta_k)$ by using Algorithm 3 ▷ Step computation
 - 5: Choose α_k ▷ Step size selection
 - 6: $\theta_{k+1} = \theta_k + \alpha_k p_k$ ▷ Iterate update
 - 7: **if** $k > S$ **then** ▷ Secant pairs update
 - 8: Discard $\{(s_{k-S}, y_{k-S})\}$ from the storage
 - 9: Compute and save $s_k = \theta_{k+1} - \theta_k$, $y_k = \nabla f(\theta_{k+1}) - \nabla f(\theta_k)$
 - 10: **end if**
 - 11: **end for**
-

7.4 Switching from first to second-order optimization

Adaptive first-order optimizers such as Adam are highly effective during the early stages of training due to their ability to handle poor gradient scaling, strong anisotropy, and stochastic noise. However, their adaptivity can bias the optimization trajectory and limit the asymptotic accuracy of the learned solution [94, 38, 28]. In contrast, quasi-Newton methods are often more effective in later stages of training, when the terms in (7) vary more smoothly along the optimization trajectory and curvature information can be exploited reliably.

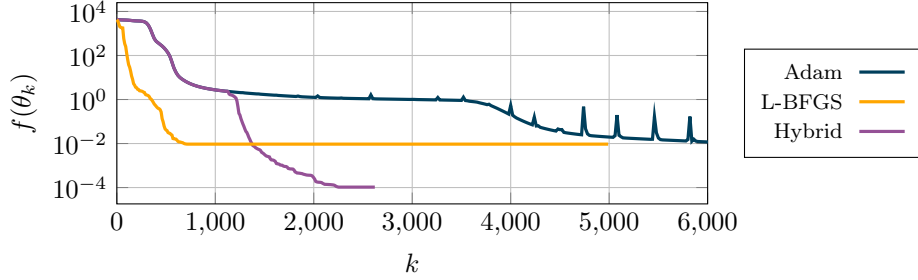


Figure 11: PINN training for Poisson problem in 1D using Adam, L-BFGS, and a hybrid Adam→L-BFGS strategy. The hybrid strategy initially follows Adam, then automatically switches to L-BFGS once the gradient norm stabilizes, yielding significantly faster convergence in the later stages.

This complementarity motivates hybrid training schedules that combine the robustness of first-order adaptive optimizers with the precision and fast local convergence of quasi-Newton methods. The most common strategy in the PINN literature is the two-stage *Adam* → *L-BFGS* scheme [71, 90]. Adam provides a robust initialization phase, bringing the parameters θ into a favorable region of the loss landscape. Once the optimization enters a smoother regime, L-BFGS then refines the solution using curvature-aware updates.

Because the optimal switching point is problem-dependent, switching too early leads to unstable curvature estimates, whereas switching too late wastes iterations. To address this, several heuristics have been proposed to determine the suitable transition point in practice. For example, one might observe that the gradient norm $\|\nabla_{\theta} f(\theta_k)\|$ is typically noisy during early Adam iterations but stabilizes once the loss landscape becomes smoother. Hence, a persistent plateau in $\|\nabla_{\theta} f(\theta_k)\|$ over several epochs indicates readiness to switch to L-BFGS [90, 59]. Alternatively, one might explore approaches based on PDE residual saturation [90, 59] or stabilization of adaptive learning rates [90, 75].

Numerical example # 5: Figure 11 illustrates the contrasting optimization behaviours of Adam, L-BFGS, and hybrid Adam→L-BFGS training using a representative PINN example (Poisson problem in 1D). As we can see, Adam often achieves rapid loss reduction during the early, noisy, and highly anisotropic phase of training, but its progress may slow down once the optimization enters a more stable regime where curvature effects dominate. Standard L-BFGS, when started from the same initialization, fails to make progress at a certain point due to unstable curvature estimates. In contrast, the hybrid Adam→L-BFGS schedule automatically transitions at the moment when the gradient norm stabilizes, enabling L-BFGS to exploit accurate curvature information and achieve substantially faster convergence as well as improved accuracy.

8 Towards scalable training for SciML: Recent advances and open challenges

In the preceding sections, we reviewed the most widely used optimization methods in SciML, ranging from first-order SGD through adaptive gradient variants to second-order techniques. In this final section, we take a step further and discuss recent trends in optimization for large-scale training of SciML models. Since much of the current research focuses on overcoming optimization stiffness and scalability limitations, our presentation adopts the preconditioning perspective and organizes existing approaches according to whether they act in data, parameter, or function space:

- *Data-space preconditioning* refers to transformations applied to the input variables or to intermediate representations prior to gradient computation. If a linear transformation M^{-1} is applied to the input x , this alters both the search direction and its magnitude, since for a model h_θ and a loss function ℓ , the sample-wise objective function becomes $f_i(\theta) = \ell(h_\theta(M^{-1}x), y)$, and its gradient is

$$\nabla_\theta f_i(\theta) = J_{h_\theta}(M^{-1}x) M^{-1} \nabla_z \ell.$$

As a result, such transformations reshape the geometry of the input distribution and can significantly affect the conditioning and convergence behaviour of the optimization process.

- *Parameter-space preconditioning* modifies the optimization dynamics by applying a transformation to the gradient as

$$\theta_{k+1} = \theta_k - \alpha_k M_k^{-1} g_k.$$

The adaptive gradient methods discussed in Section 3, which construct diagonal preconditioners M_k by maintaining estimates of the gradient’s moments, fit this framework.

- *Function-space preconditioning* refers to transformations that modify the geometry of the optimization problem directly in the space of functions represented by the DNN, rather than in input or parameter space. This viewpoint is natural for SciML, where the learning objective is often defined through a differential operator $\mathcal{N}$. Let u denote the DNN output. For each sample, the objective is given by $f(u) = \ell(\mathcal{N}(u), q)$, and the functional gradient reads

$$\nabla_u f = \mathcal{N}^*(\mathcal{N}(u) - q).$$

A function-space preconditioner applies an operator $\mathcal{M}^{-1}$ to this gradient:

$$u_{k+1} = u_k - \alpha_k \mathcal{M}^{-1} \nabla_u f,$$

where ideally $\mathcal{M} \approx \mathcal{N}^* \mathcal{N}$. This mirrors classical PDE preconditioning and motivates several architectural and algorithmic strategies in SciML. However, unlike in linear PDE solvers, the operator $\mathcal{N}$ is composed with a nonlinear neural representation, making the design and analysis of effective function-space preconditioners substantially more challenging [14].

All these techniques aim to improve the conditioning of the underlying optimization problem. In what follows, we review how preconditioning strategies developed for classical ML are applied in SciML, highlight their limitations, and discuss physics-oriented preconditioners specifically tailored to SciML. While SciML can be viewed as a particular instance of classical ML, the interaction with physical constraints makes the behavior of these methods less well understood. Given the breadth and rapid evolution of this field, we do not attempt a comprehensive survey, but instead provide a structured overview of selected research directions and open challenges.

8.1 Data-space preconditioning

Classical data-space preconditioning techniques in classical ML that are also widely used by SciML practitioners include feature scaling and standardisation [98], data augmentation [55], and equation non-dimensionalisation [90]. However, in SciML settings, these ubiquitous techniques often lack a rigorous theoretical foundation. For instance, in contrast to standard ML, normalisation may interact in subtle ways with the PDE structure, or the constraint enforcement, potentially leading to undesirable optimization dynamics [36].

Other input transformations specifically designed for SciML have been motivated by physical considerations. In particular, input representations that enrich the coordinate system by introducing high-frequency components have been successfully proposed to mitigate the spectral bias of DNNs. Prominent examples include Fourier feature mappings [84], positional encodings [69], or graph embeddings [56]. While these techniques substantially accelerate the learning of oscillatory or multiscale solutions, the choice of frequencies, bandwidths, or embedding structures remains largely problem-dependent and empirical [70, 91].

In the context of PINNs, *sampling strategies* also play a central role in determining how residuals and BC influence the conditioning of the underlying optimization problem (see Section 5). Consequently, many advanced sampling strategies can be interpreted as data-space preconditioners. Popular examples include variance-reduced and importance sampling strategies [66, 13], adaptive and residual-based sampling [95], or failure-informed or active sampling approaches [20]. Similarly, *adaptive weighting* strategies for balancing loss terms play a key role in stable and efficient PINN training (see Section 2.2.2). Existing methods dynamically adjust weights based on gradient magnitudes, NTK spectra, or residual statistics to alleviate stiffness and imbalance during optimization (e.g., [60, 83, 5, 92]). Despite their practical effectiveness, these sampling and weighting strategies are largely heuristic in nature.

8.2 Parameter-space preconditioning

While established optimization techniques such as AdaGrad [18], RMSProp [85], and Adam [39] are computationally efficient and robust in many large-scale ML applications, their behaviour in SciML settings remains poorly understood. In particular, recent studies have reported gradient pathologies and unexpected optimizer dynamics in stiff PDE-constrained problems, suggesting nontrivial interactions between adaptive scaling and PDE-induced spectral properties [76].

Other classically used preconditioned optimization methods attempt to approximate the inverse Hessian or Fisher information matrix. The algorithms such as natural gradient descent [2], K-FAC [58], Shampoo [26], or SOAP [89] maintain structured curvature approximations, often exploiting Kronecker or layerwise factorizations to remain tractable. However, despite their awareness of curvature, these approaches remain limited in large-scale settings due to substantial memory and computational costs [97].

Specific to SciML are the preconditioners that leverage its link with numerical linear algebra and scientific computing. Several recent approaches adapt large-scale preconditioning techniques from PDE solvers to DNN training, with a particular emphasis on *domain-decomposition and multilevel ideas*. These methods provide implicit preconditioning by restricting updates to lower-dimensional, better-conditioned parameter subspaces, which is especially appealing in SciML, where parameter groups often align with physical modes or operator components. In domain-decomposition approaches [41], the parameter space is often partitioned into blocks (e.g., layers, channels, Fourier modes, or operator components), and optimization proceeds via additive or multiplicative Schwarz-type iterations; see, e.g., [43, 48, 25, 3, 80]. Despite the promising empirical performance of these methods, principled guidelines for constructing stable parameter decompositions and convergence guarantees in stochastic regimes remain largely absent, with several recent developments making progress in this direction [22].

Closely related multilevel approaches equip the parameter space with a hierarchy of coarse-to-fine DNN representations, analogous to mesh hierarchies in multigrid methods [86]. Coarse networks, obtained for example by reducing network width, depth, or spectral resolution, provide inexpensive approximations of curvature and enable the construction of coarse search directions that can accelerate convergence [19, 44, 8, 1, 68, 7]. Here, we note that the design of effective coarse spaces, inter-level transfer operators, and robust multilevel optimization schemes for DNNs remains an open problem, particularly in the presence of stochastic gradients, with recent developments encompassing multi-level AdaGrad [23] and multilevel adaptive regularization techniques [57].

8.3 Function-space preconditioning

One example of function-space preconditioning is Sobolev training [12], which augments the loss with derivative information, thereby modifying the inner product in function space. In the SciML context, variants of this idea have been developed to guide PINNs toward solutions that minimize errors in Sobolev

norms, often improving convergence rates and robustness relative to standard ℓ_2 losses [82]. Nevertheless, the choice of derivative orders, relative weightings, and their interaction with PDE stiffness and BC conditions remains largely experimental, and a systematic theory linking Sobolev loss design to operator spectra and optimization conditioning still remains an active area of research.

As in parameter-space preconditioning, several recent approaches draw inspiration from multilevel and domain-decomposition methods. In the context of PINNs, domain-decomposition variants explicitly partition the physical domain and assign separate DNNs to each subregion, as in XPINNs [34], cPINNs [35], FBPINNs [65], or APINNs [30]. From a parameter-space perspective, assigning separate networks effectively decomposes the global parameter vector into weakly coupled blocks, yielding an implicit block-diagonal preconditioner that improves conditioning and stabilizes optimization. Simultaneously, by restructuring the functional basis, these methods enhance conditioning, enable parallelization, and facilitate the representation of localised or multiscale features, often allowing PDE problems that are otherwise inaccessible to monolithic PINNs to be solved. It is worth noting that the effectiveness of these methods depends critically on the choice of subdomain partitioning, interface conditions, and coupling strategies, for which systematic design principles remain largely open.

Similarly, several recent multilevel strategies construct and utilize hierarchies of function spaces to train SciML models more efficiently. By first learning a coarse approximation and then refining the function space, these approaches reduce spectral bias and accelerate convergence. Representative examples include multilevel training for data-driven models [54] and multifidelity PINNs [72, 61], block-structured coarse-to-fine training [24], and multilevel domain-decomposition methods [17], while related multiresolution and wavelet- or Fourier-based architectures [27] achieve similar effects by separating low- and high-frequency components. Despite the significant empirical success of these multilevel strategies, theoretical guidance for selecting coarse spaces, designing restriction and prolongation operators, and ensuring convergence, particularly in stochastic training regimes, remains only partially understood.

8.4 Outlook

The approaches reviewed in this section illustrate how modern SciML training increasingly draws on principles from classical numerical PDE solvers while retaining the flexibility of DNNs. Data-space, parameter-space, and function-space preconditioning address complementary aspects of the optimization challenge and together provide a unified perspective for mitigating stiffness, multiscale behaviour, and poor conditioning in large-scale SciML training. Nevertheless, many open questions remain, particularly in nonconvex and stochastic settings, and the relationships between these three viewpoints are still only partially understood. As a result, the design of efficient optimization techniques remains an active area of research, with continued progress needed to develop scalable and reliable methods for increasingly complex SciML applications.

Acknowledgements

The work of A.K. benefited from Artificial and Natural Intelligence Toulouse Institute (ANITI), funded by the France 2030 program under Grant Agreement No. ANR-23-IACL-0002. The work of E.R. was supported by the ANR project MEPHISTO (ANR-24-CE23-7039) and PEPR IA SHARP.

References

- [1] S. AHAMED, N. ZAKARIAEI, E. HABER, AND M. ELIASOF, *Multiscale training of convolutional neural networks*, arXiv e-prints, (2025), pp. arXiv–2501.
- [2] S.-I. AMARI, *Natural gradient works efficiently in learning*, Neural Computation, 10 (1998), pp. 251–276.
- [3] E. AMID, R. ANIL, AND M. WARMUTH, *Locoprop: Enhancing backprop via local loss optimization*, in International conference on artificial intelligence and statistics, PMLR, 2022, pp. 9626–9642.
- [4] H. ANAGNOSTOPOULOS, M. PAPACHRISTOU, D. GIOVANIS, AND P. KOUMOUTSAKOS, *Residual-based adaptive reweighting for physics-informed neural networks*, Journal of Computational Physics, 478 (2023), p. 111959.
- [5] S. J. ANAGNOSTOPOULOS, J. D. TOSCANO, N. STERGIOPULOS, AND G. E. KARNIADAKIS, *Residual-based attention in physics-informed neural networks*, Computer Methods in Applied Mechanics and Engineering, 421 (2024), p. 116805.
- [6] L. BOTTOU, F. E. CURTIS, AND J. NOCEDAL, *Optimization methods for large-scale machine learning*, Siam Review, 60 (2018), pp. 223–311.
- [7] H. CALANDRA, S. GRATTON, E. RICCIETTI, AND X. VASSEUR, *On a multilevel Levenberg–Marquardt method for the training of artificial neural networks and its application to the solution of partial differential equations*, Optimization Methods and Software, 37 (2022), pp. 361–386.
- [8] B. CHANG, L. MENG, E. HABER, F. TUNG, AND D. BEGERT, *Multi-level residual networks from dynamical systems view*, arXiv preprint arXiv:1710.10348, (2017).
- [9] R. T. CHEN, Y. RUBANOVA, J. BETTENCOURT, AND D. K. DUVENAUD, *Neural ordinary differential equations*, Advances in neural information processing systems, 31 (2018).
- [10] T. CHEN, L. LU, AND G. KARNIADAKIS, *BRDR: Boundary residual decay reweighting for physics-informed neural networks*, Computer Methods in Applied Mechanics and Engineering, 418 (2024), p. 116731.

- [11] Z. CHEN, V. BADRINARAYANAN, C.-Y. LEE, AND A. RABINOVICH, *Grad-norm: Gradient normalization for adaptive loss balancing in deep multitask networks*, in International conference on machine learning, PMLR, 2018, pp. 794–803.
- [12] W. M. CZARNECKI, S. OSINDERO, M. JADERBERG, G. SWIRSZCZ, AND R. PASCANU, *Sobolev training for neural networks*, Advances in neural information processing systems, 30 (2017).
- [13] A. DAW, J. BU, S. WANG, P. PERDIKARIS, AND A. KARPATNE, *Rethinking the importance of sampling in physics-informed neural networks*, arXiv preprint arXiv:2207.02338, (2022).
- [14] T. DE RYCK, F. BONNET, S. MISHRA, AND E. DE BÉZENAC, *An operator preconditioning perspective on training in physics-informed machine learning*, arXiv preprint arXiv:2310.05801, (2023).
- [15] R. S. DEMBO, S. C. EISENSTAT, AND T. STEIHAUG, *Inexact newton methods*, SIAM Journal on Numerical analysis, 19 (1982), pp. 400–408.
- [16] J. DICK, F. Y. KUO, AND I. H. SLOAN, *High-dimensional integration: the quasi-Monte Carlo way*, Acta Numerica, 22 (2013), pp. 133–288.
- [17] V. DOLEAN, A. HEINLEIN, S. MISHRA, AND B. MOSELEY, *Multilevel domain decomposition-based architectures for physics-informed neural networks*, Computer Methods in Applied Mechanics and Engineering, 429 (2024), p. 117116.
- [18] J. DUCHI, E. HAZAN, AND Y. SINGER, *Adaptive subgradient methods for online learning and stochastic optimization.*, Journal of machine learning research, 12 (2011).
- [19] L. GAEDKE-MERZHÄUSER*, A. KOPANIČÁKOVÁ*, AND R. KRAUSE, *Multilevel minimization for deep residual networks*, in Proceedings of French-German-Swiss Optimization Conference (FGS’2019), 2021.
- [20] Z. GAO, L. YAN, AND T. ZHOU, *Failure-informed adaptive sampling for pinns*, SIAM Journal on Scientific Computing, 45 (2023), pp. A1971–A1994.
- [21] G. GARRIGOS AND R. M. GOWER, *Handbook of convergence theorems for (stochastic) gradient methods*, ArXiv preprint 2301.11235, (2023).
- [22] S. GRATTON, A. KOPANIČÁKOVÁ, AND P. TOINT, *Recursive bound-constrained AdaGrad with applications to multilevel and domain decomposition minimization*, arXiv preprint arXiv:2507.11513, (2025).
- [23] S. GRATTON, A. KOPANIČÁKOVÁ, AND P. L. TOINT, *Multilevel objective-function-free optimization with an application to neural networks training*, SIAM Journal on Optimization, 33 (2023), pp. 2772–2800.

- [24] S. GRATTON, V. MERCIER, E. RICCIETTI, AND P. L. TOINT, *A block-coordinate approach of multi-level optimization with an application to physics-informed neural networks*, Computational Optimization and Applications, 89 (2024), pp. 385–417.
- [25] L. GU, W. ZHANG, J. LIU, AND X.-C. CAI, *Decomposition and composition of deep convolutional neural networks and training acceleration via sub-network transfer learning*, (2022).
- [26] V. GUPTA, T. KOREN, AND Y. SINGER, *Shampoo: Preconditioned stochastic tensor optimization*, in International Conference on Machine Learning, PMLR, 2018, pp. 1842–1850.
- [27] W. HE, J. LI, X. KONG, AND L. DENG, *Multi-level physics informed deep learning for solving partial differential equations in computational structural mechanics*, Communications Engineering, 3 (2024), p. 151.
- [28] M. HEUSEL ET AL., *GANs trained by a two time-scale update rule converge to a local nash equilibrium*, in NeurIPS, 2017.
- [29] R. A. HORN AND C. R. JOHNSON, *Matrix analysis*, Cambridge university press, 2012.
- [30] Z. HU, A. D. JAGTAP, G. E. KARNIADAKIS, AND K. KAWAGUCHI, *Augmented Physics-Informed Neural Networks (APINNs): A gating network-based soft domain decomposition methodology*, Engineering Applications of Artificial Intelligence, 126 (2023), p. 107183.
- [31] B. IRWIN AND E. HABER, *Secant penalized BFGS: a noise robust quasi-Newton method via penalizing the secant condition*, Computational Optimization and Applications, 84.3 (2023), pp. 651–702.
- [32] A. JACOT, F. GABRIEL, AND C. HONGLER, *Neural tangent kernel: Convergence and generalization in neural networks*, NeurIPS, (2018).
- [33] A. D. JAGTAP AND G. E. KARNIADAKIS, *Adaptive activation functions accelerate convergence in physics-informed neural networks*, Journal of Computational Physics, 404 (2020), p. 109136.
- [34] ———, *Extended physics-informed neural networks (XPINNs): A generalized space-time domain decomposition based deep learning framework for nonlinear partial differential equations*, Communications in Computational Physics, 28 (2020).
- [35] A. D. JAGTAP, E. KHARAZMI, AND G. E. KARNIADAKIS, *Conservative physics-informed neural networks on discrete domains for conservation laws: Applications to forward and inverse problems*, Computer Methods in Applied Mechanics and Engineering, 365 (2020), p. 113028.

- [36] G. E. KARNIADAKIS, I. G. KEVREKIDIS, L. LU, P. PERDIKARIS, S. WANG, AND L. YANG, *Physics-informed machine learning*, Nature Reviews Physics, 3 (2021), pp. 422–440.
- [37] A. KENDALL, Y. GAL, AND R. CIPOLLA, *Multi-task learning using uncertainty to weigh losses for scene geometry and semantics*, CVPR, (2018), pp. 7482–7491.
- [38] N. S. KESKAR, D. MUDIGERE, J. NOCEDAL, M. SMELYANSKIY, AND P. T. P. TANG, *On large-batch training for deep learning: Generalization gap and sharp minima*, ICLR, (2017).
- [39] D. P. KINGMA AND J. BA, *Adam: A method for stochastic optimization*, 2017.
- [40] E. KIYANI, K. SHUKLA, J. F. URBÁN, J. DARBON, AND G. E. KARNIADAKIS, *Optimizing the optimizer for physics-informed neural networks and Kolmogorov-Arnold networks*, Computer Methods in Applied Mechanics and Engineering, 446 (2025), p. 118308.
- [41] A. KLAWONN, M. LANSER, AND J. WEBER, *Machine learning and domain decomposition methods-a survey*, Computational Science and Engineering, 1 (2024), p. 2.
- [42] D. A. KNOLL AND D. E. KEYES, *Jacobian-free Newton–Krylov methods: a survey of approaches and applications*, Journal of Computational Physics, 193 (2004), pp. 357–397.
- [43] A. KOPANIČÁKOVÁ, H. KOTHARI, G. E. KARNIADAKIS, AND R. KRAUSE, *Enhancing training of physics-informed neural networks using domain decomposition-based preconditioning strategies*, SIAM Journal on Scientific Computing, 46 (2024), pp. S46–S67.
- [44] A. KOPANIČÁKOVÁ AND R. KRAUSE, *Globally Convergent Multilevel Training of Deep Residual Networks*, SIAM Journal on Scientific Computing, 45 (2023), pp. S254–S280.
- [45] A. KRISHNAPRIYAN, A. GHOLAMI, S. ZHE, R. M. KIRBY, AND M. W. MAHONEY, *Characterizing possible failure modes in physics-informed neural networks*, Advances in Neural Information Processing Systems (NeurIPS), 34 (2021), pp. 26548–26560.
- [46] M. LAZZARA, M. CHEVALIER, M. COLOMBO, J. G. GARCIA, C. LAPEYRE, AND O. TESTE, *Surrogate modelling for an aircraft dynamic landing loads simulation using an lstm autoencoder-based dimensionality reduction approach*, Aerospace Science and Technology, 126 (2022), p. 107629.
- [47] J. LEE, L. XIAO, S. SCHOENHOLZ, Y. BAHRI, R. NOVAK, J. SOHL-DICKSTEIN, AND J. PENNINGTON, *Wide neural networks of any depth evolve as linear models under gradient descent*, in Advances in Neural Information Processing Systems (NeurIPS), 2019.

- [48] Y. LEE, A. KOPANIČÁKOVÁ, AND G. E. KARNIADAKIS, *Two-level overlapping additive Schwarz preconditioner for training scientific machine learning applications*, Computer Methods in Applied Mechanics and Engineering, 448 (2026), p. 118400.
- [49] H. LI, Z. XU, G. TAYLOR, C. STUDER, AND T. GOLDSTEIN, *Visualizing the loss landscape of neural nets*, Advances in neural information processing systems, 31 (2018).
- [50] D. C. LIU AND J. NOCEDAL, *On the limited memory BFGS method for large scale optimization*, Mathematical Programming, 45 (1989), pp. 503–528.
- [51] I. LOSHCHILOV AND F. HUTTER, *SGDR: Stochastic gradient descent with warm restarts*, in ICLR, 2017.
- [52] L. LU, P. JIN, G. PANG, Z. ZHANG, AND G. E. KARNIADAKIS, *Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators*, Nature Machine Intelligence, 3 (2021), pp. 218–229.
- [53] L. LU, X. MENG, Z. MAO, AND G. E. KARNIADAKIS, *DeepXDE: A deep learning library for solving differential equations*, SIAM review, 63 (2021), pp. 208–228.
- [54] K. O. LYE, S. MISHRA, AND R. MOLINARO, *A multi-level procedure for enhancing accuracy of machine learning algorithms*, European Journal of Applied Mathematics, 32 (2021), pp. 436–469.
- [55] K. MAHARANA, S. MONDAL, AND B. NEMADE, *A review: Data pre-processing and data augmentation techniques*, Global Transitions Proceedings, 3 (2022), pp. 91–99.
- [56] I. MAKAROV, D. KISELEV, N. NIKITINSKY, AND L. SUBELJ, *Survey on graph embeddings and their applications to machine learning problems on graphs*, PeerJ Computer Science, 7 (2021), p. e357.
- [57] F. MARINI, M. PORCELLI, AND E. RICCIETTI, *A multilevel stochastic regularized first-order method with application to training*, arXiv preprint arXiv:2412.11630, (2024).
- [58] J. MARTENS AND R. GROSSE, *Optimizing neural networks with Kronecker-factored approximate curvature*, in International conference on machine learning, PMLR, 2015, pp. 2408–2417.
- [59] L. MCCLENNY AND U. BRAGA-NETO, *Self-adaptive physics-informed neural networks using a residual-based adaptive activation*, arXiv:2009.04544, (2020).
- [60] L. D. MCCLENNY AND U. M. BRAGA-NETO, *Self-adaptive physics-informed neural networks*, Journal of Computational Physics, 474 (2023), p. 111722.

- [61] X. MENG AND G. E. KARNIADAKIS, *A composite neural network that learns from multi-fidelity data: Application to function approximation and inverse pde problems*, Journal of Computational Physics, 401 (2020), p. 109020.
- [62] S. MISHRA, *A machine learning framework for data driven acceleration of computations of differential equations*, arXiv preprint arXiv:1807.09519, (2018).
- [63] S. MISHRA AND R. MOLINARO, *Estimates on the generalization error of physics-informed neural networks for approximating pdes*, IMA Journal of Numerical Analysis, 43 (2023), pp. 1–43.
- [64] R. MOLINARO AND S. MISHRA, *Learning with uncertainty in PINNs: Bayesian physics-informed neural networks*, Computer Methods in Applied Mechanics and Engineering, 397 (2022), p. 115153.
- [65] B. MOSELEY, A. MARKHAM, AND T. NISSEN-MEYER, *Finite basis physics-informed neural networks (FBPINNs): a scalable domain decomposition approach for solving differential equations*, Advances in Computational Mathematics, 49 (2023), p. 62.
- [66] M. A. NABIAN, R. J. GLADSTONE, AND H. MEIDANI, *Efficient training of physics-informed neural networks via importance sampling*, Computer-Aided Civil and Infrastructure Engineering, 36 (2021), pp. 962–977.
- [67] J. NOCEDAL AND S. WRIGHT, *Numerical optimization*, Springer Science & Business Media, 2006.
- [68] C. PONCE, R. LI, C. MAO, AND P. VASSILEVSKI, *Multilevel-in-width training for deep neural network regression*, Numerical Linear Algebra with Applications, 30 (2023), p. e2501.
- [69] A. QUARTERONI, P. GERVASIO, AND F. REGAZZONI, *Combining physics-based and data-driven models: advancing the frontiers of research with scientific machine learning*, arXiv preprint arXiv:2501.18708, (2025).
- [70] N. RAHAMAN, A. BARATIN, D. ARPIT, F. DRAXLER, M. LIN, F. HAMPRECHT, Y. BENGIO, AND A. COURVILLE, *On the spectral bias of neural networks*, in International conference on machine learning, PMLR, 2019, pp. 5301–5310.
- [71] M. RAISSI, P. PERDIKARIS, AND G. KARNIADAKIS, *Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations*, Journal of Computational Physics, 378 (2019), pp. 686–707.
- [72] E. RICCIETTI, V. MERCIER, S. GRATTON, AND P. BOUDIER, *Multilevel physics informed neural networks (mpinns)*, in The International Conference on Learning Representations (ICLR), 2022, pp. 1–15.

- [73] H. ROBBINS AND S. MONRO, *A stochastic approximation method*, The annals of mathematical statistics, (1951), pp. 400–407.
- [74] F. ROOSTA-KHORASANI AND M. W. MAHONEY, *Sub-sampled Newton methods*, Mathematical Programming, 174 (2019), pp. 293–326.
- [75] A. RUIZ, C. OSORIO, J. SANCHEZ, C. E. PACHON, AND A. ÖZISARI, *When physics meets deep learning: Training dynamics and diagnostic tools for physics-informed neural networks*, Journal of Computational Physics, 463 (2022), p. 111262.
- [76] M. SCOTT, T. XU, Z. TANG, A. PICHETTE-EMMONS, Q. YE, Y. SAAD, AND Y. XI, *Designing preconditioners for sgd: Local conditioning, noise floors, and basin stability*, arXiv preprint arXiv:2511.19716, (2025).
- [77] S. SHALEV-SHWARTZ AND S. BEN-DAVID, *Understanding Machine Learning: From Theory to Algorithms*, Cambridge University Press, 2014.
- [78] C. J. SHALLUE, J. LEE, J. ANTOGNINI, ET AL., *Measuring the effects of data parallelism on neural network training*, JMLR, (2019).
- [79] H.-J. M. SHI, Y. XIE, R. BYRD, AND J. NOCEDAL, *A noise-tolerant quasi-Newton algorithm for unconstrained optimization*, SIAM Journal on Optimization, 32 (2022), pp. 29–55.
- [80] J. W. SIEGEL, Q. HONG, X. JIN, W. HAO, AND J. XU, *Greedy training algorithms for neural networks and applications to pdes*, Journal of Computational Physics, 484 (2023), p. 112084.
- [81] S. L. SMITH, P.-J. KINDERMANS, AND Q. V. LE, *Don’t decay the learning rate, increase the batch size*, in ICLR, 2018.
- [82] H. SON, J. W. JANG, W. J. HAN, AND H. J. HWANG, *Sobolev training for physics informed neural networks*, arXiv preprint arXiv:2101.08932, (2021).
- [83] S. SUBRAMANIAN, R. M. KIRBY, M. W. MAHONEY, AND A. GHOLAMI, *Adaptive self-supervision algorithms for physics-informed neural networks*, arXiv preprint arXiv:2207.04084, (2022).
- [84] M. TANCIK, P. SRINIVASAN, B. MILDENHALL, S. FRIDOVICH-KEIL, N. RAGHAVAN, U. SINGHAL, R. RAMAMOORTHY, J. BARRON, AND R. NG, *Fourier features let networks learn high frequency functions in low dimensional domains*, Advances in neural information processing systems, 33 (2020), pp. 7537–7547.
- [85] T. TIELEMAN, *Lecture 6.5-rmsprop: Divide the gradient by a running average of its recent magnitude*, COURSERA: Neural networks for machine learning, 4 (2012), p. 26.
- [86] U. TROTTEMBERG, C. W. OOSTERLEE, AND A. SCHULLER, *Multigrid*, Elsevier, 2000.

- [87] V. VAPNIK, *Statistical Learning Theory*, Wiley, 1998.
- [88] R. VERFÜRTH, *A posteriori error estimation and adaptive mesh-refinement techniques*, Journal of Computational and Applied Mathematics, 50 (1994), pp. 67–83.
- [89] N. VYAS, D. MORWANI, R. ZHAO, M. KWUN, I. SHAPIRA, D. BRANDFONBRENER, L. JANSON, AND S. KAKADE, *Soap: Improving and stabilizing Shampoo using Adam*, arXiv preprint arXiv:2409.11321, (2024).
- [90] S. WANG, S. SANKARAN, AND P. PERDIKARIS, *An expert’s guide to training physics-informed neural networks*, Journal of Computational Physics, 473 (2023), p. 111742.
- [91] S. WANG, Y. TENG, AND P. PERDIKARIS, *Understanding and mitigating gradient flow pathologies in physics-informed neural networks*, SIAM Journal on Scientific Computing, 43 (2021), pp. A3055–A3081.
- [92] S. WANG, Y. TENG, AND P. PERDIKARIS, *When and why PINNs fail to train: A neural tangent kernel perspective*, Journal of Computational Physics, 449 (2022), p. 110768.
- [93] S. WANG, H. WANG, AND P. PERDIKARIS, *On the eigenvector bias of Fourier feature networks: From regression to solving multi-scale PDEs with physics-informed neural networks*, Computer Methods in Applied Mechanics and Engineering, 384 (2021), p. 113938.
- [94] A. C. WILSON, R. ROELOFS, M. STERN, N. SREBRO, AND B. RECHT, *The marginal value of adaptive gradient methods in machine learning*, NeurIPS, (2017).
- [95] C. WU, M. ZHU, Q. TAN, Y. KARTHA, AND L. LU, *A comprehensive study of non-adaptive and residual-based adaptive sampling for physics-informed neural networks*, Computer Methods in Applied Mechanics and Engineering, 403 (2023), p. 115671.
- [96] L. XIANG, L. PENG, X. CHEN, AND G. LIN, *lbPINN: A likelihood-based physics-informed neural network*, Journal of Computational Physics, 451 (2021), p. 110829.
- [97] S. ZAMPINI, U. ZERBINATI, G. TURKYIAH, AND D. KEYES, *PETScML: Second-order solvers for training regression problems in Scientific Machine Learning*, in Proceedings of the Platform for Advanced Scientific Computing Conference, 2024, pp. 1–12.
- [98] H. ZHONGKAI, J. YAO, C. SU, H. SU, Z. WANG, F. LU, Z. XIA, Y. ZHANG, S. LIU, L. LU, ET AL., *Pinnacle: A comprehensive benchmark of physics-informed neural networks for solving pdes*, Advances in Neural Information Processing Systems, 37 (2024), pp. 76721–76774.

- [99] L. ZHU, S. WANG, AND P. PERDIKARIS, *Adaptive sampling for PINNs using uncertainty quantification*, Computer Methods in Applied Mechanics and Engineering, 402 (2023), p. 115720.